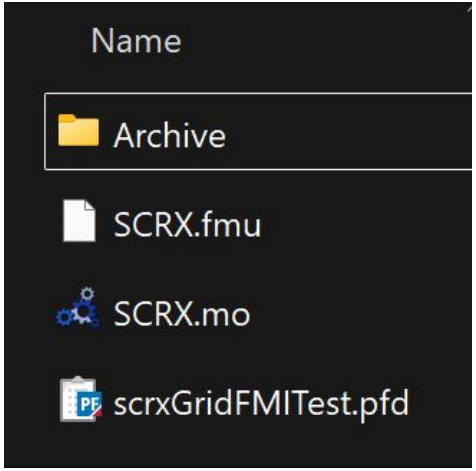
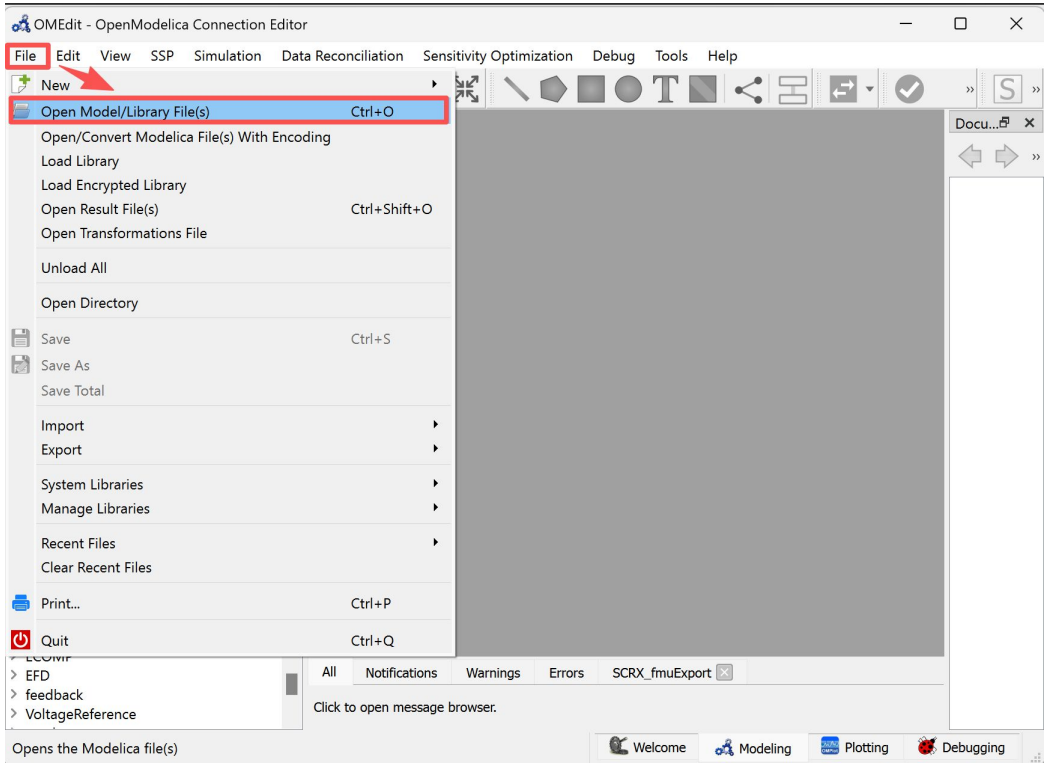


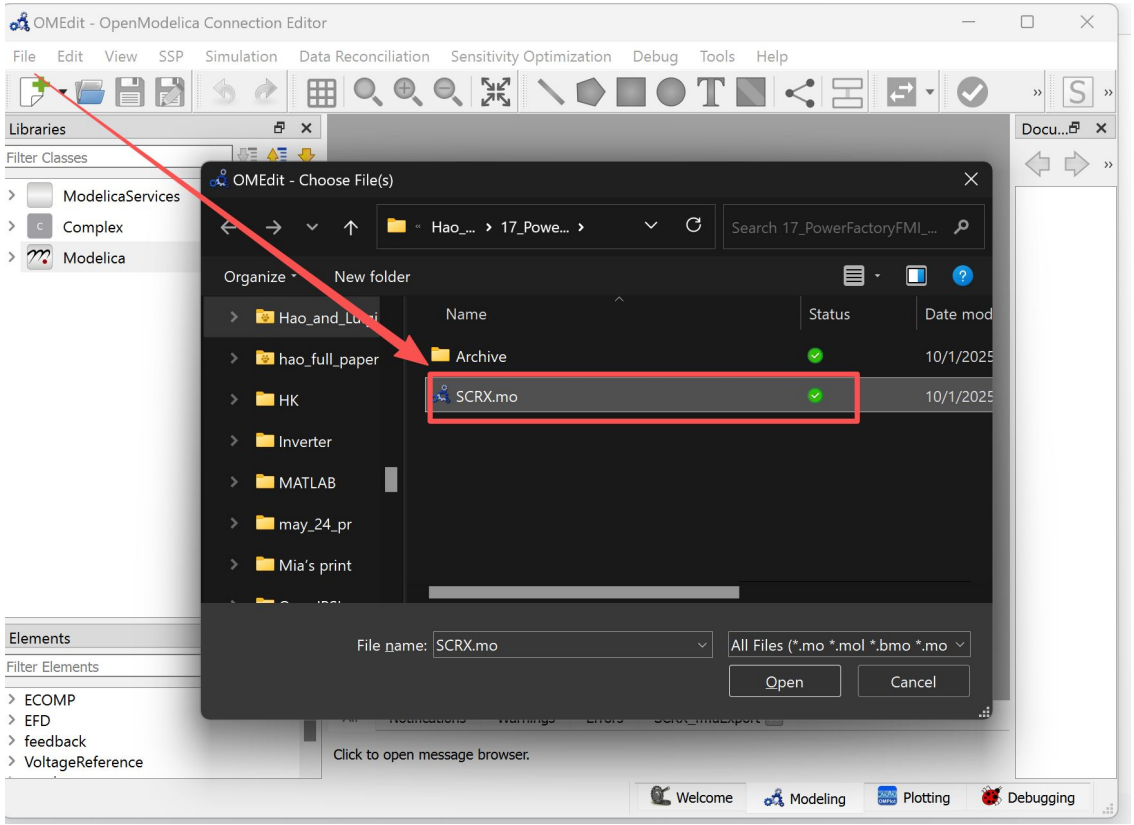
This page serves as the **cover and introduction** to the tutorial. It visually highlights the connection between **PowerFactory (PF)** and the **Functional Mock-up Interface (FMI)** standard. The icons at the top indicate the tools we will use, while the file list below (`SCRX.fmu`, `SCRX.mo`, and `scrxGridFMITest.pfd`) represents the core components of the example project.

The description should clarify for the reader:

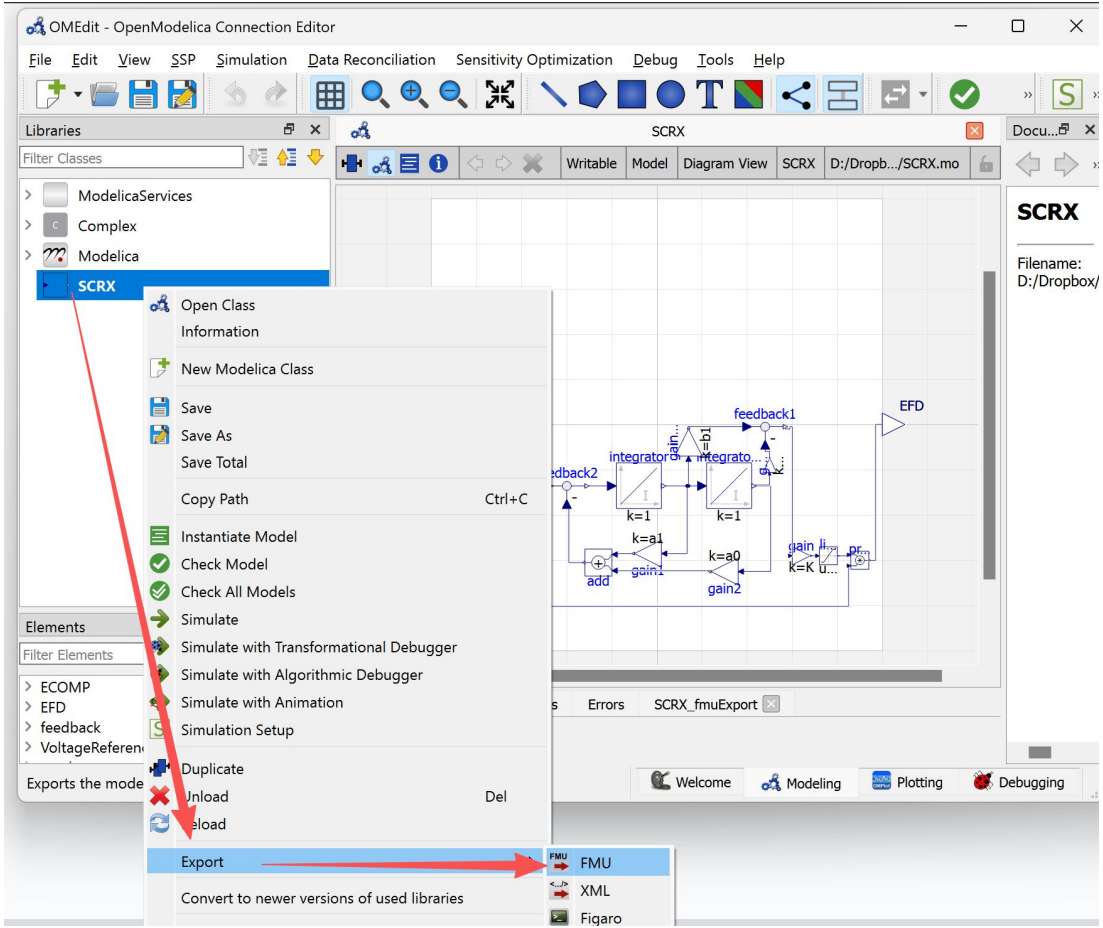
- This tutorial will walk through how to use FMI models within PowerFactory.
- The example files shown here (FMU package, Modelica model, and PowerFactory project file) will be used in later steps.
- By the end, readers will understand how to **import, configure, and simulate FMI models in PowerFactory**.



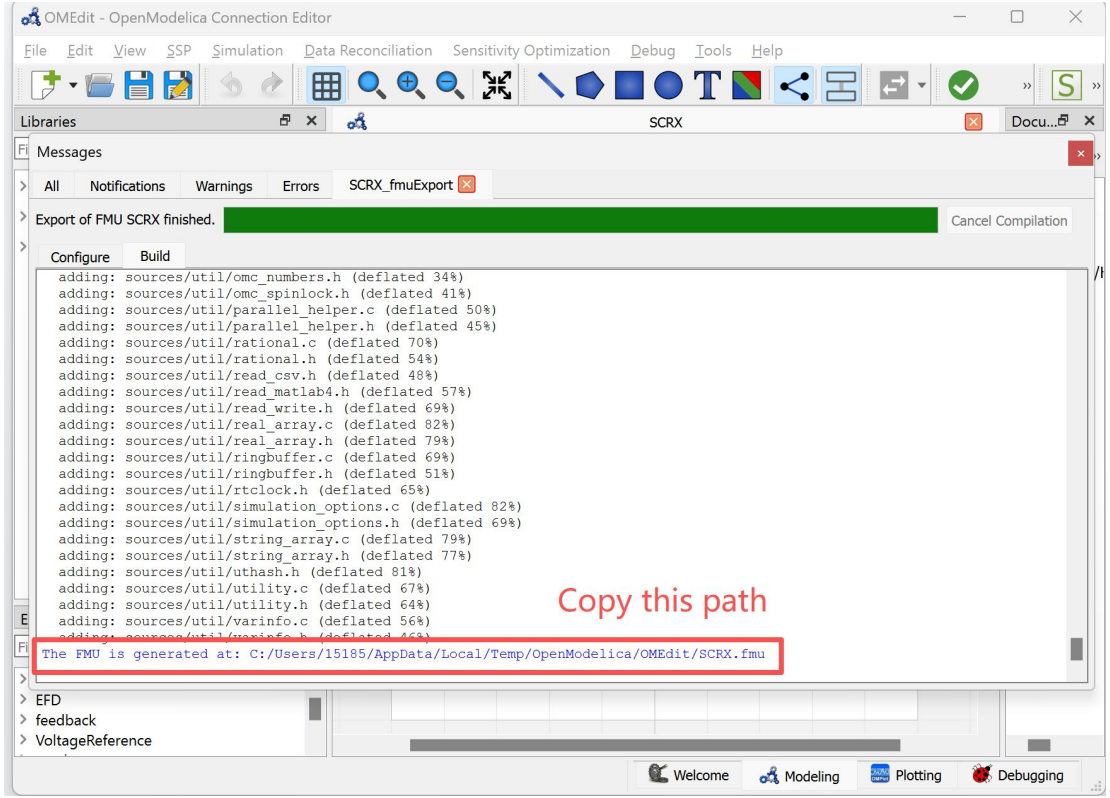
This page shows how to begin working with the Modelica file that will later be exported as an FMU for use in PowerFactory. In **OMEdit (OpenModelica Connection Editor)**, navigate to **File → Open Model/Library File(s)** (or use the shortcut `Ctrl+O`) to load the `.mo` file.



After choosing **File → Open Model/Library File(s)** in OMEdit, this step shows how to locate and load the specific Modelica model that will be used in the tutorial. In this case, the file **`SCRX.mo`** is selected from the project directory.

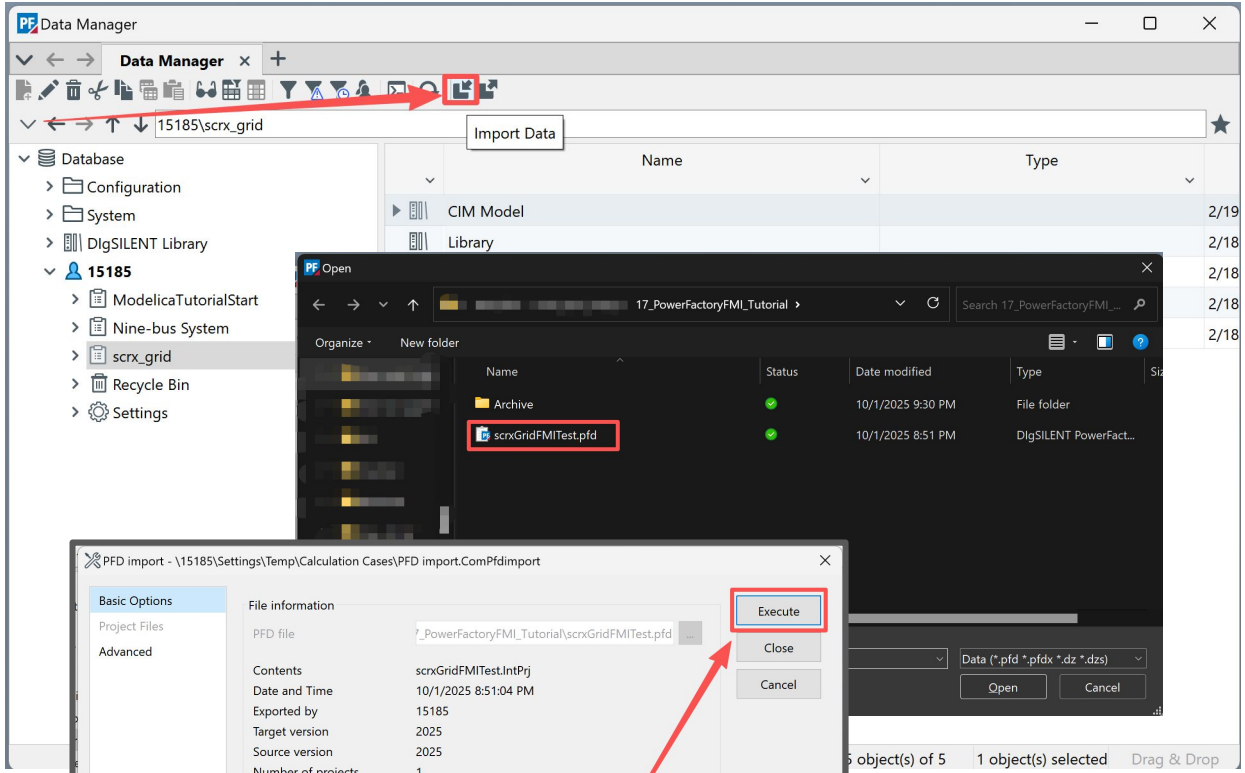


With the Modelica file loaded, the next step is to export it as a **Functional Mock-up Unit (FMU)**. In OMEdit, right-click on the model (SCRX in this case), then select **Export** → **FMU** from the context menu.



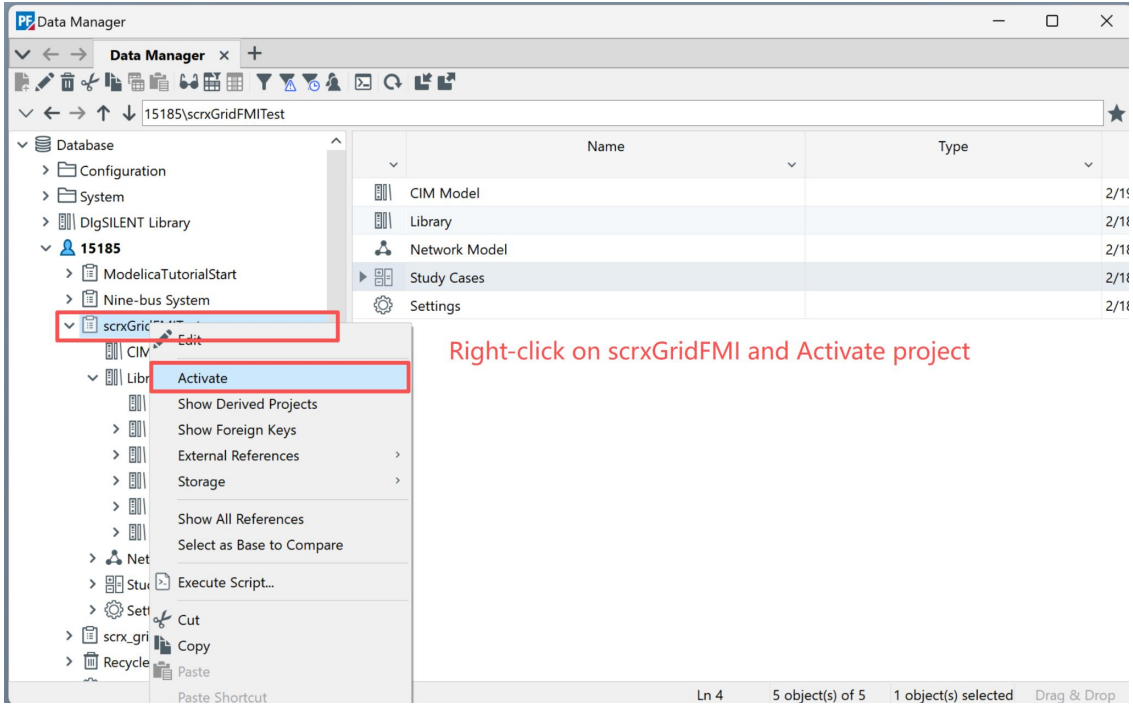
Once the export process finishes, OMEdit confirms that the FMU has been successfully generated. A green progress bar indicates completion, and the **output log** provides the location of the exported file.

It is important to **copy this path** or move the FMU file to your working directory for easier access later. This FMU package will be imported into PowerFactory in the next steps of the tutorial.



In PowerFactory's Data Manager, create or select a database folder (here named 15185/scrx_grid) and choose Import Data. From the file selection window, locate the provided project file scrxGridFMITest.pfd.

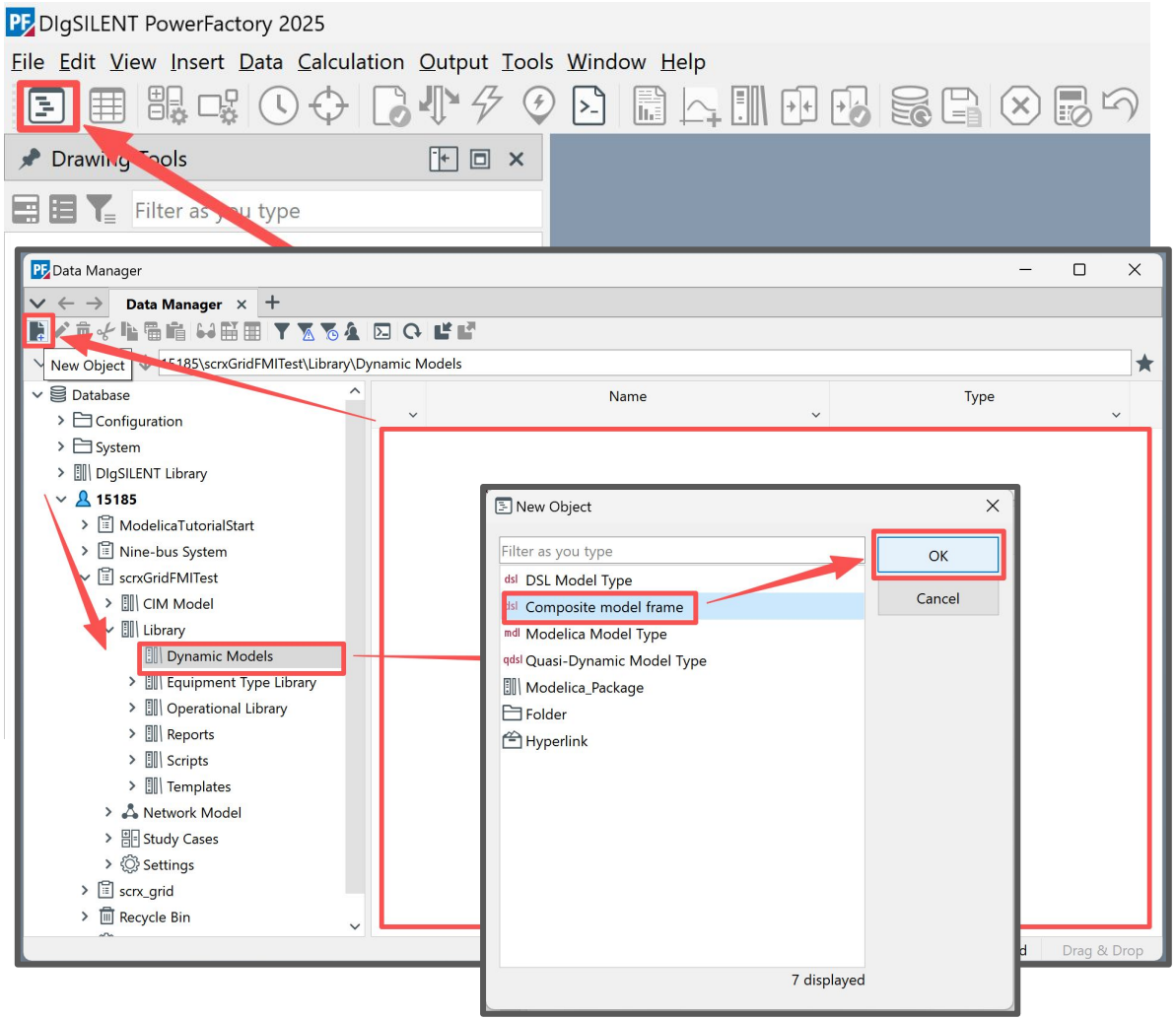
After selecting the file, confirm the import options in the PFD Import dialog and click Execute to load the project. Once imported, the case will appear in the Data Manager and can be activated for further work.



Right-click on scrxGridFMI and Activate project

In PowerFactory's Data Manager, create or select a database folder (here named 15185/scrx_grid) and choose Import Data. From the file selection window, locate the provided project file scrxGridFMITest.pfd.

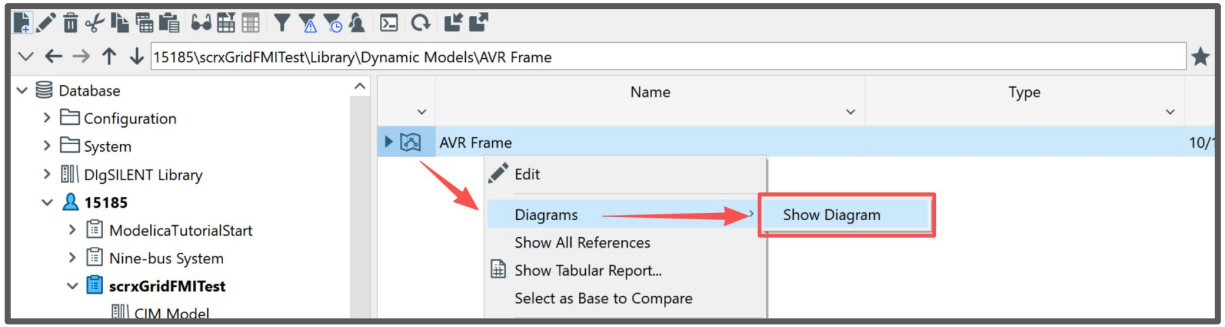
After selecting the file, confirm the import options in the PFD Import dialog and click Execute to load the project. Once imported, the case will appear in the Data Manager and can be activated for further work.

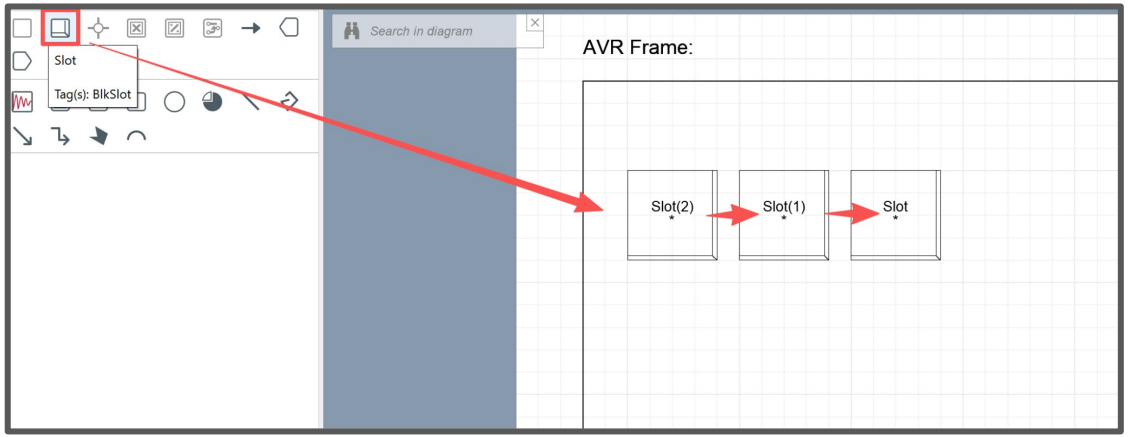


To prepare PowerFactory for integrating the FMU, create a new Composite Model Frame inside the project's Dynamic Models library.

1. In the Data Manager, navigate to: Library → Dynamic Models
2. Click the New Object button in the toolbar.
3. In the New Object dialog, select Composite model frame and click OK.

The Composite Model Frame acts as a container that will later be linked to the FMU and mapped to elements in the network model. This step establishes the foundation for embedding external FMI-based models into PowerFactory's dynamic simulation environment.

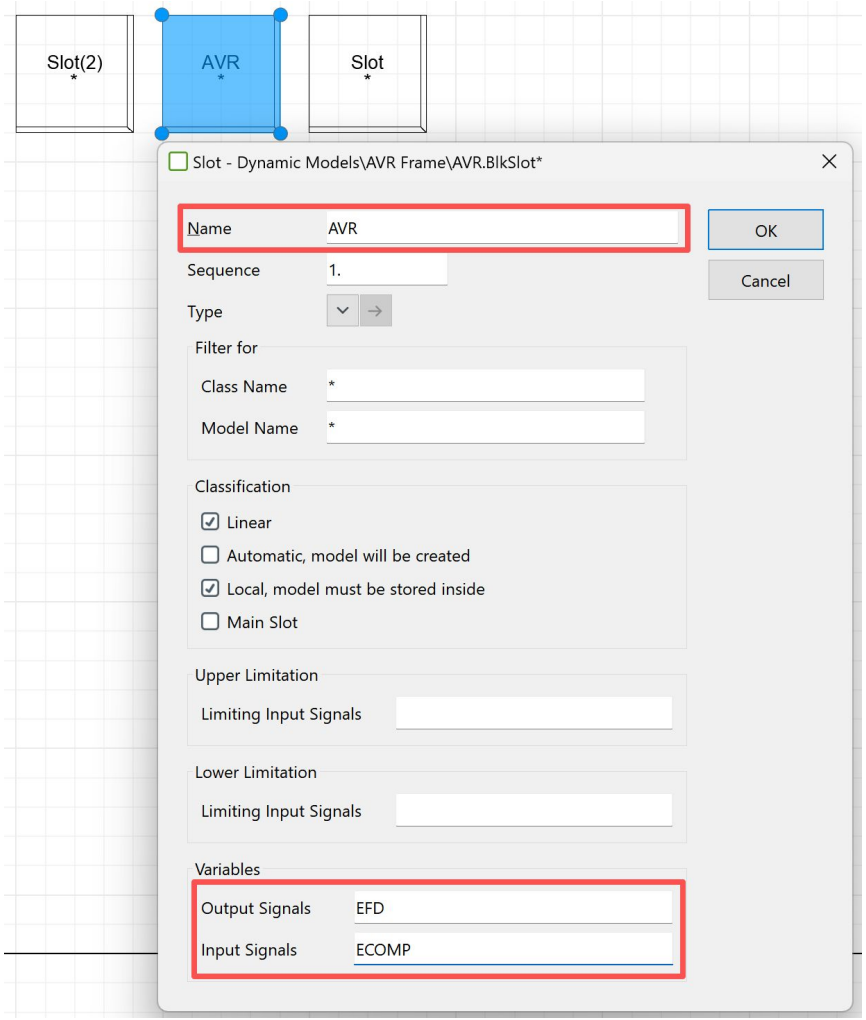




After creating the Composite Model Frame, give it a clear name (here: AVR Frame) and confirm with OK. This creates a new dynamic model container inside the project library.

1. Right-click on the new object (AVR Frame) and select Show Diagram.
2. In the diagram editor, add Slots from the toolbar. Slots act as placeholders for model connections (inputs/outputs) and will later be mapped to the FMU interface.
3. Arrange the slots in sequence to represent the expected signal flow.

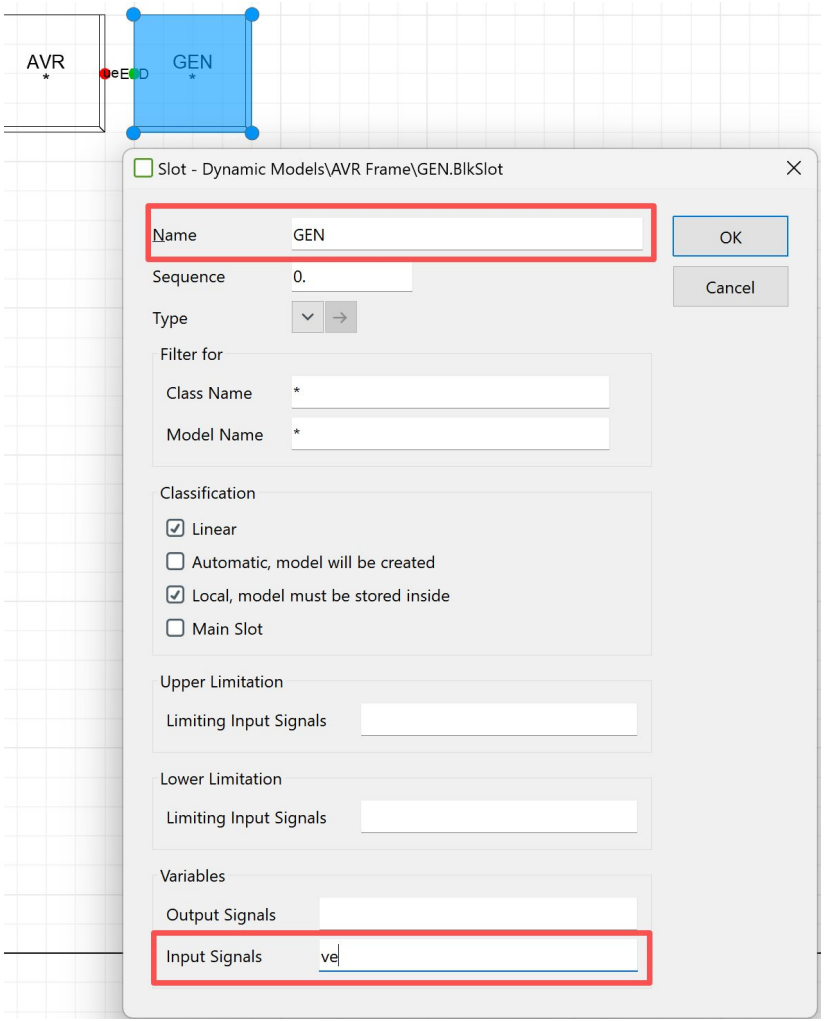
This step establishes the framework where the FMU will be integrated, ensuring that the signals can be correctly routed between the FMU and PowerFactory's simulation environment.



Inside the **Composite Model Frame** diagram, each slot must be configured to represent the correct role in the control structure.

1. Double-click on the slot and assign it a **Name** (here: *AVR*).
2. Define the **Input Signals** and **Output Signals**:
   - **Input Signal**: *ECOMP*
   - **Output Signal**: *EFD*
3. Keep relevant options selected (e.g., *Linear*) to ensure proper signal handling.

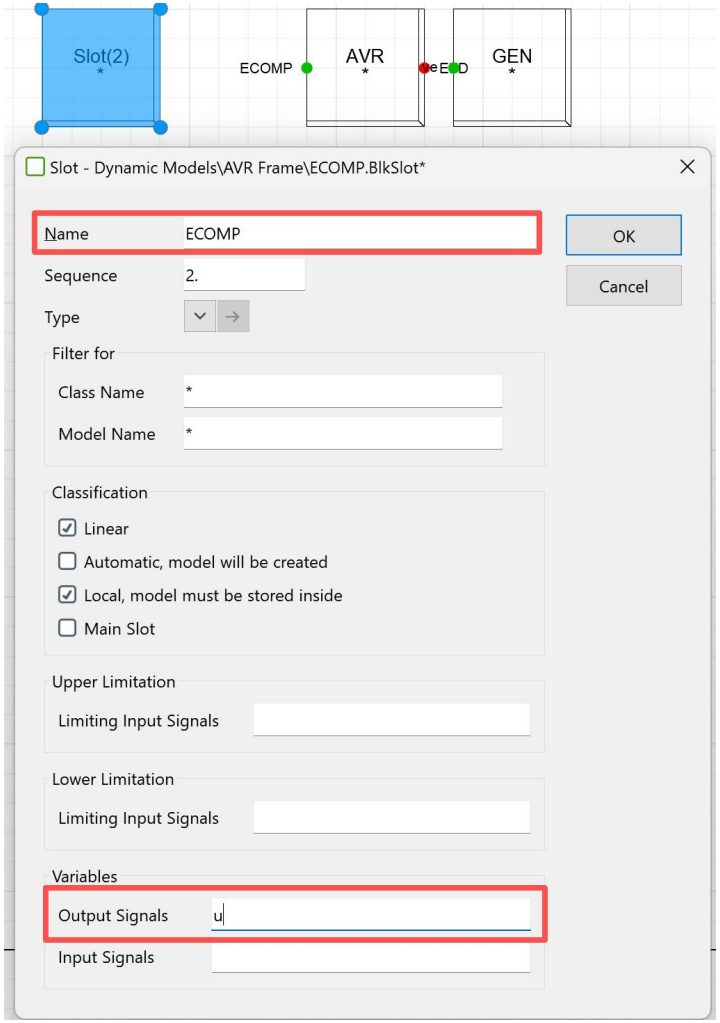
This step links the abstract slot object to the intended control variables, enabling PowerFactory to map them to FMU ports later. Properly naming and assigning inputs/outputs is critical for ensuring the FMI model integrates smoothly into the grid simulation.

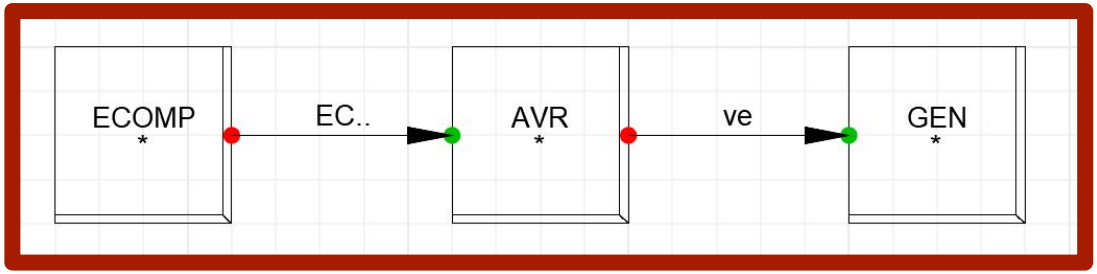


In addition to the AVR slot, a **Generator (GEN) slot** is created in the Composite Model Frame to represent the machine interface.

1. Double-click the new slot and set its **Name** to *GEN*.
2. Define the **Input Signals** (here: *ve*, representing the voltage input).
3. Keep the relevant classification settings (e.g., *Linear*) enabled.

This slot establishes the connection point for the generator within the control loop. By combining the **AVR slot** (controller) with the **GEN slot** (plant), the composite model can exchange signals with the FMU in a structured way, ensuring proper integration during simulation.





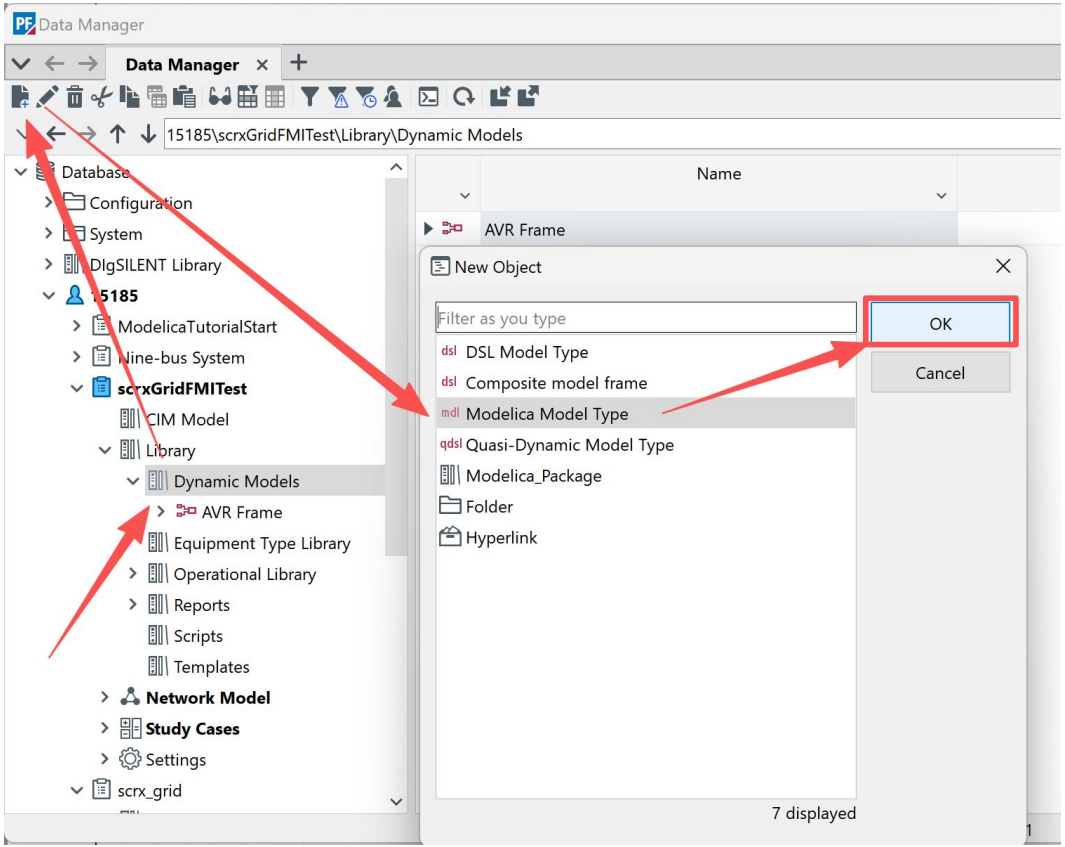
The last slot to configure in the composite model is the ECOMP (excitation control input).

1. Double-click the slot and set the Name to ECOMP.
2. Define the Output Signals (here: u).
3. Ensure the classification options remain consistent with the other slots (e.g., Linear).

With the ECOMP slot added, the signal flow is now complete:

- ECOMP → AVR → GEN

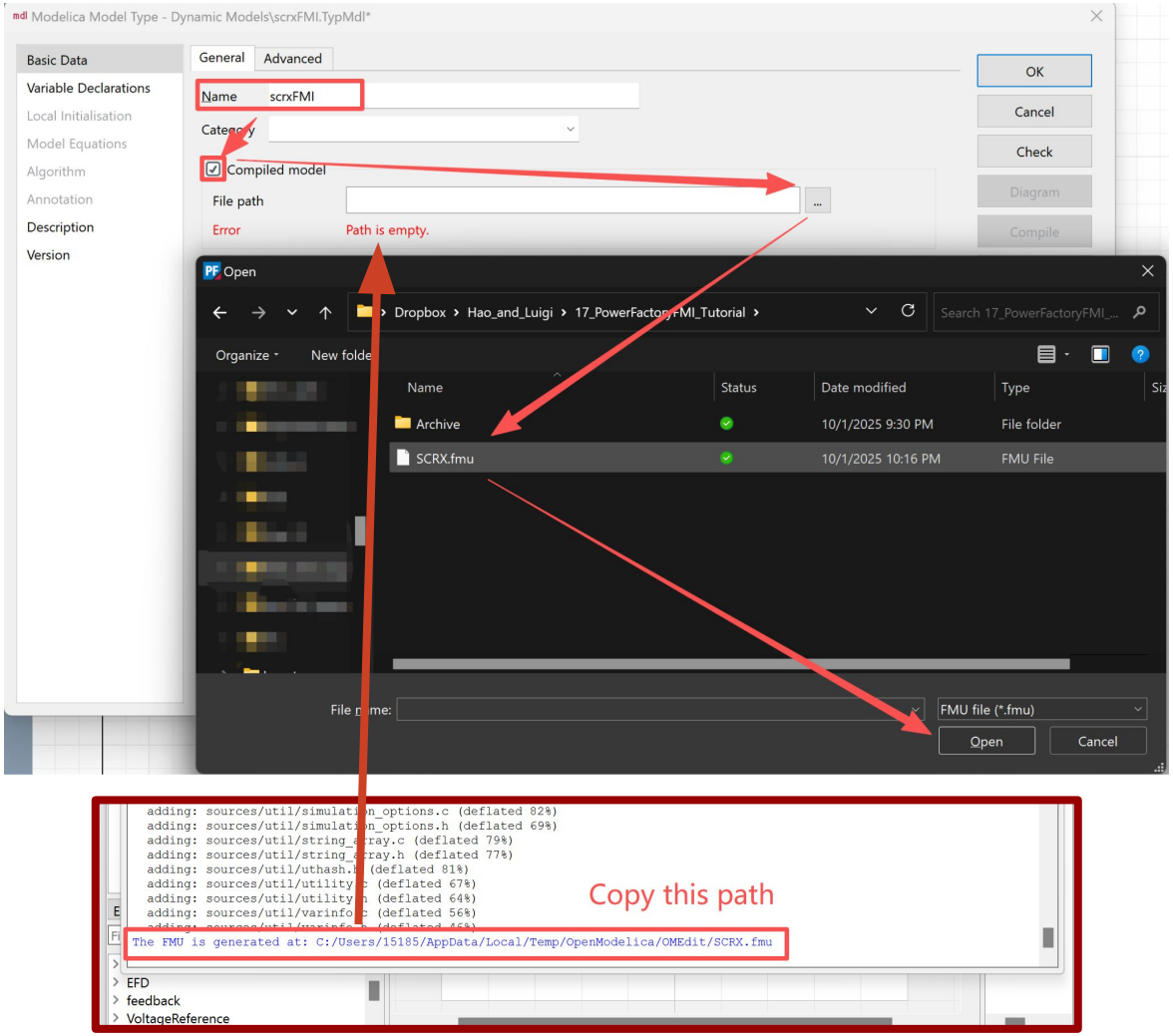
This structure mirrors the logical flow of the excitation system: ECOMP provides the input, the AVR processes and regulates the signal, and the GEN slot represents the generator receiving the final control input.



With the Composite Model Frame prepared, the next step is to add a **Modelica Model Type** inside the **Dynamic Models** library.

1. In the **Data Manager**, navigate to: `Library → Dynamic Models`
2. Click **New Object** from the toolbar.
3. In the dialog, select **Modelica Model Type** and click **OK**.

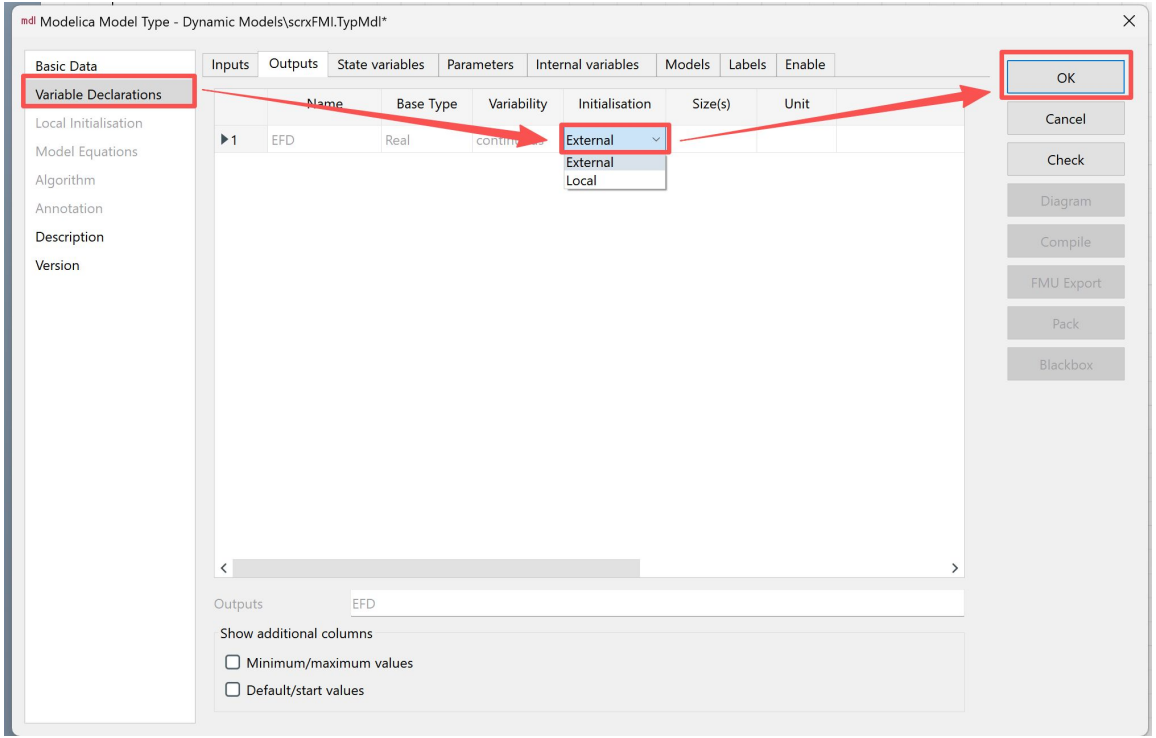
The Modelica Model Type object serves as the actual container for importing and linking the FMU into PowerFactory. Later, this object will be referenced inside the Composite Frame to bind the FMU model with the defined slots and signal connections.



In the **Modelica Model Type** dialog, configure the object to use the exported FMU:

1. Enter a descriptive **Name** (e.g., *scrxFMI*).
2. Check **Compiled model** to indicate that this object references an external FMU rather than native Modelica code.
3. Under **File path**, browse to the FMU generated earlier (e.g., `SCRX.fmu`). You can also copy the path directly from the OpenModelica export log.

This step establishes the formal link between PowerFactory and the external FMU. Once saved, this Modelica Model Type can be assigned to slots in the Composite Model Frame, enabling the FMU to exchange signals with the network model.

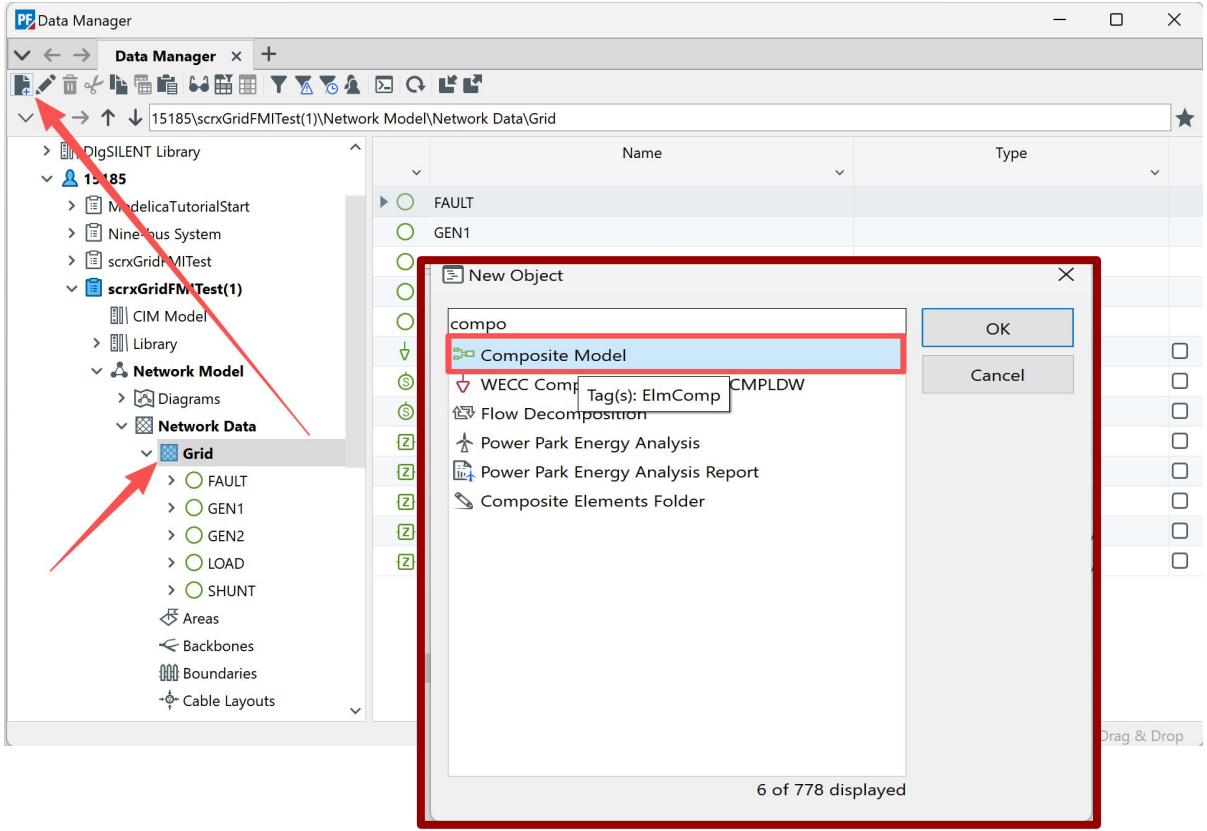


After linking the FMU, the next step is to define its variables inside the **Modelica Model Type** object:

1. Open the **Variable Declarations** tab.
2. Add the FMU variables (e.g., *EFD* as an output).
3. Set the **Initialisation** field to **External**, indicating that this variable will be provided by the FMU rather than defined locally in PowerFactory.

4. Confirm by clicking **OK**.

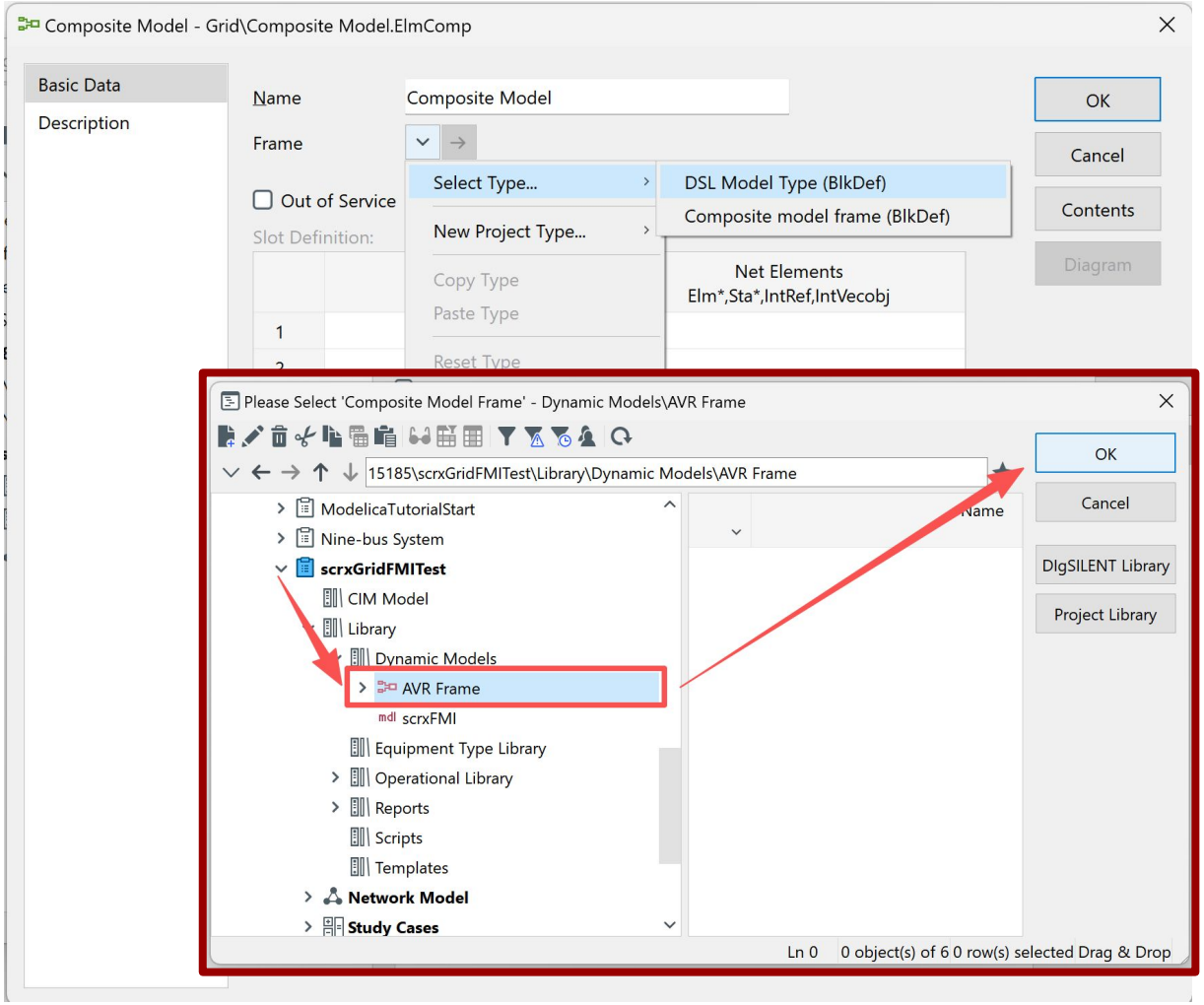
Declaring variables as *External* ensures that PowerFactory correctly maps inputs and outputs between the FMU and the composite frame slots. This step is essential for signal exchange during dynamic simulations.



To integrate the FMU-based model into the grid, create a **Composite Model** object in the **Network Data** section:

1. In the **Data Manager**, navigate to: `Network Model → Network Data → Grid`.
2. Click the **New Object** button.
3. From the dialog, select **Composite Model** and confirm with **OK**.

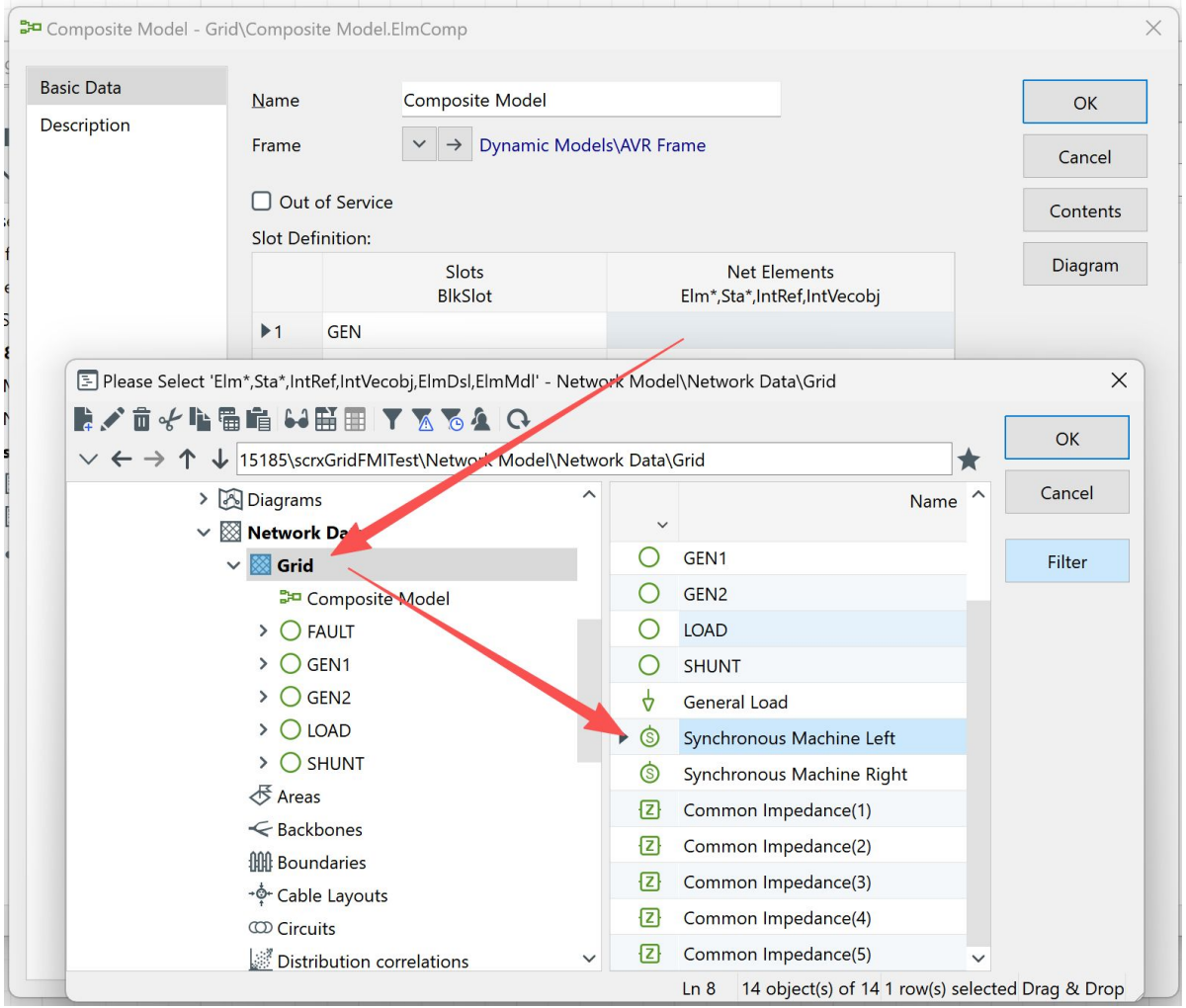
The Composite Model created here acts as the actual network element that will reference the **Composite Model Frame** and the **FMU-backed Modelica Model Type** defined earlier. This step bridges the gap between the abstract dynamic model library and the physical grid structure.



After creating the **Composite Model** in the grid, you need to link it to the previously defined **Composite Model Frame**.

1. In the Composite Model properties, select **Frame → Composite model frame (BlkDef)**.
2. Browse through the library and select the correct frame (here: *AVR Frame*).
3. Confirm with **OK**.

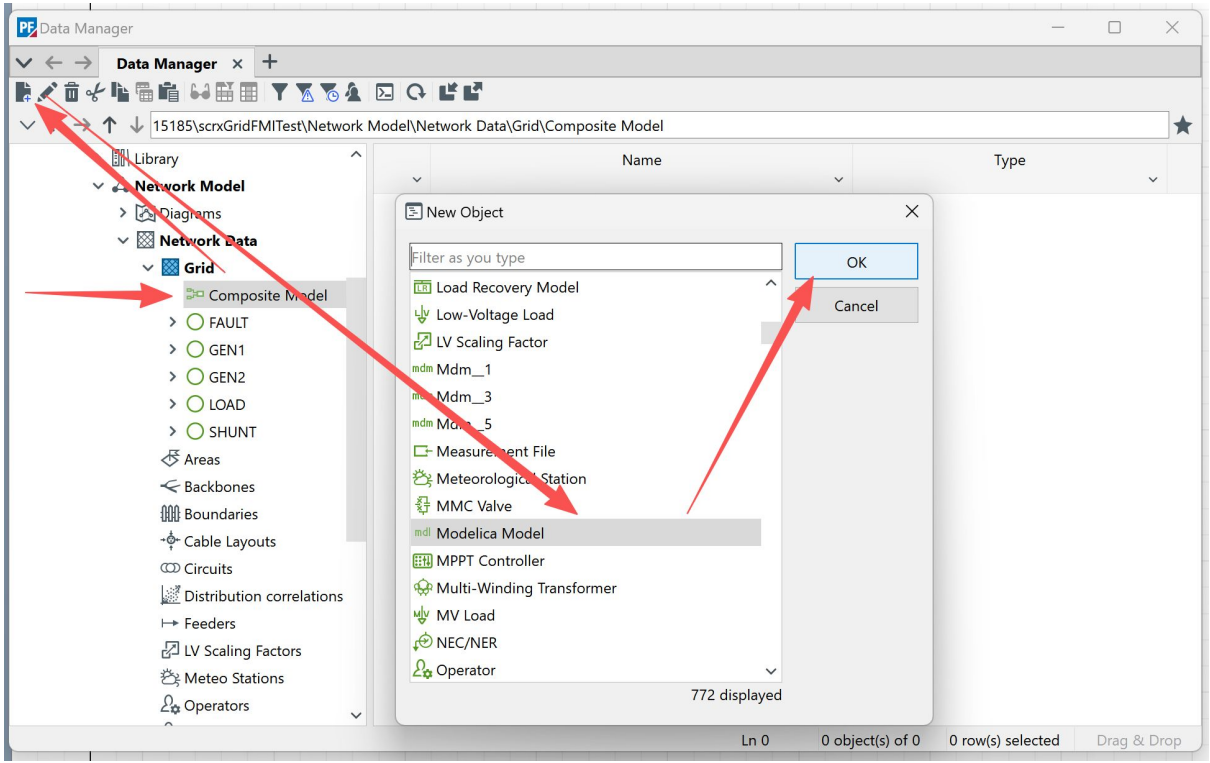
By assigning the Composite Model Frame, the network-level object is now associated with the slot structure you designed earlier. This ensures that signals exchanged in the simulation will follow the defined slot connections and map correctly to the FMU.



After assigning the **Composite Model Frame**, the next step is to map each slot to the corresponding element in the network.

1. In the **Composite Model** properties, select the slot (e.g., *GEN*).
2. From the **Net Elements** list, choose the associated grid element — in this case, a **Synchronous Machine (GEN1)**.
3. Confirm with **OK**.

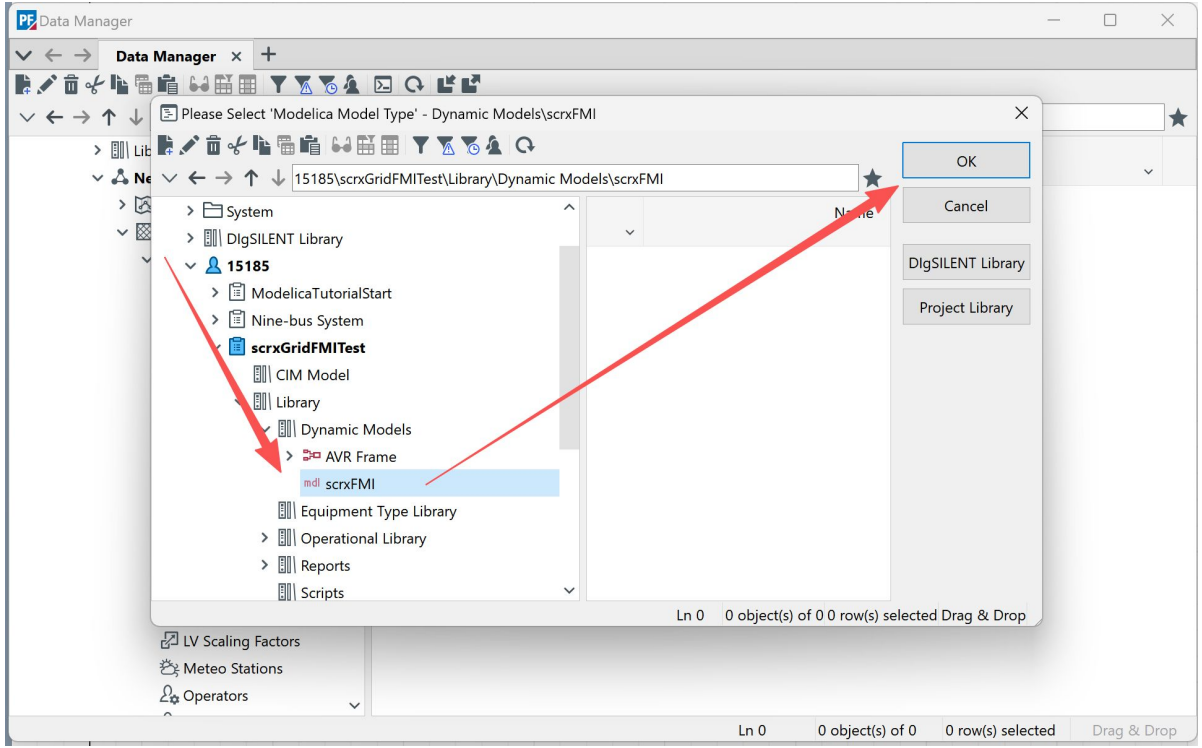
This mapping links the slot placeholders defined in the composite frame (e.g., AVR, GEN, ECOMP) to the actual components in the PowerFactory network model. With this step, the FMU model is fully integrated into the simulation environment and ready to exchange signals with real grid elements.



With the **Composite Model** created and mapped to the grid, the next step is to insert the FMU-backed Modelica object.

1. In the **Composite Model** under *Network Data → Grid*, click **New Object**.
2. From the dialog, select **Modelica Model** and confirm with **OK**.

This Modelica Model object represents the FMU within the network-level composite model. It provides the actual implementation that will interact with the slots (AVR, GEN, ECOMP) defined earlier, completing the connection between the FMU and the grid simulation environment.

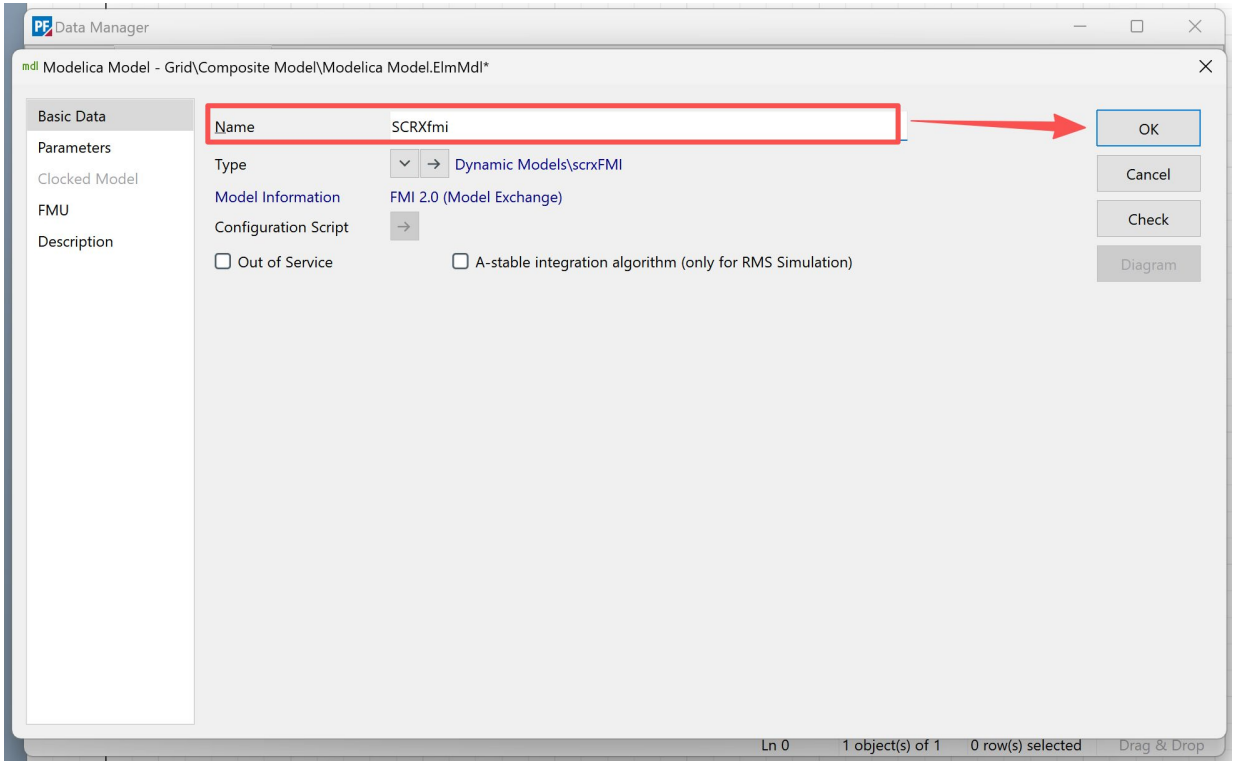


Once the **Modelica Model** object is created inside the Composite Model, you need to assign it to the correct **Modelica Model Type**.

1. In the assignment dialog, navigate to the library path where the FMU-backed type was defined (here: *scrxFMI*).

2. Select the FMU-linked Modelica Model Type and click **OK**.

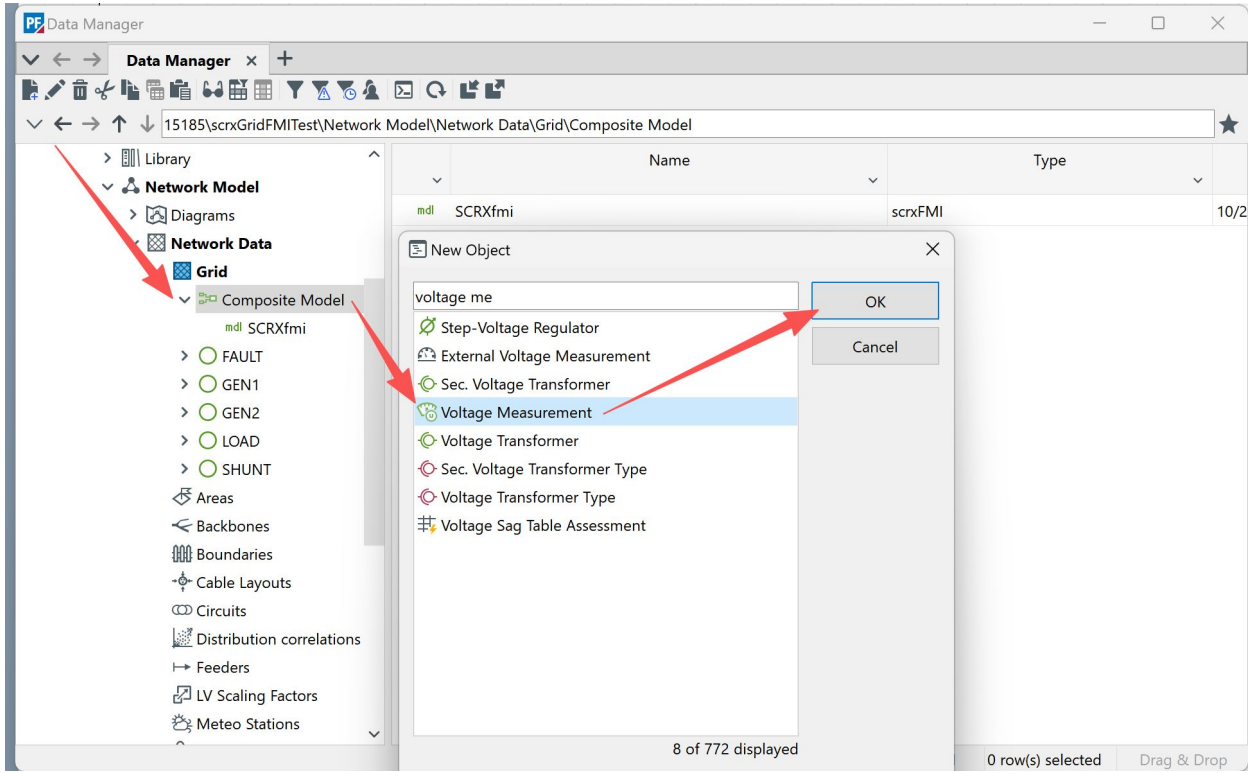
This step binds the network-level Modelica Model to the FMU you previously imported. With this link, the FMU becomes an active component inside the Composite Model and can exchange signals with the defined slots (ECOMP, AVR, GEN) during dynamic simulation.



In the **Modelica Model** properties, complete the setup of the FMU connection:

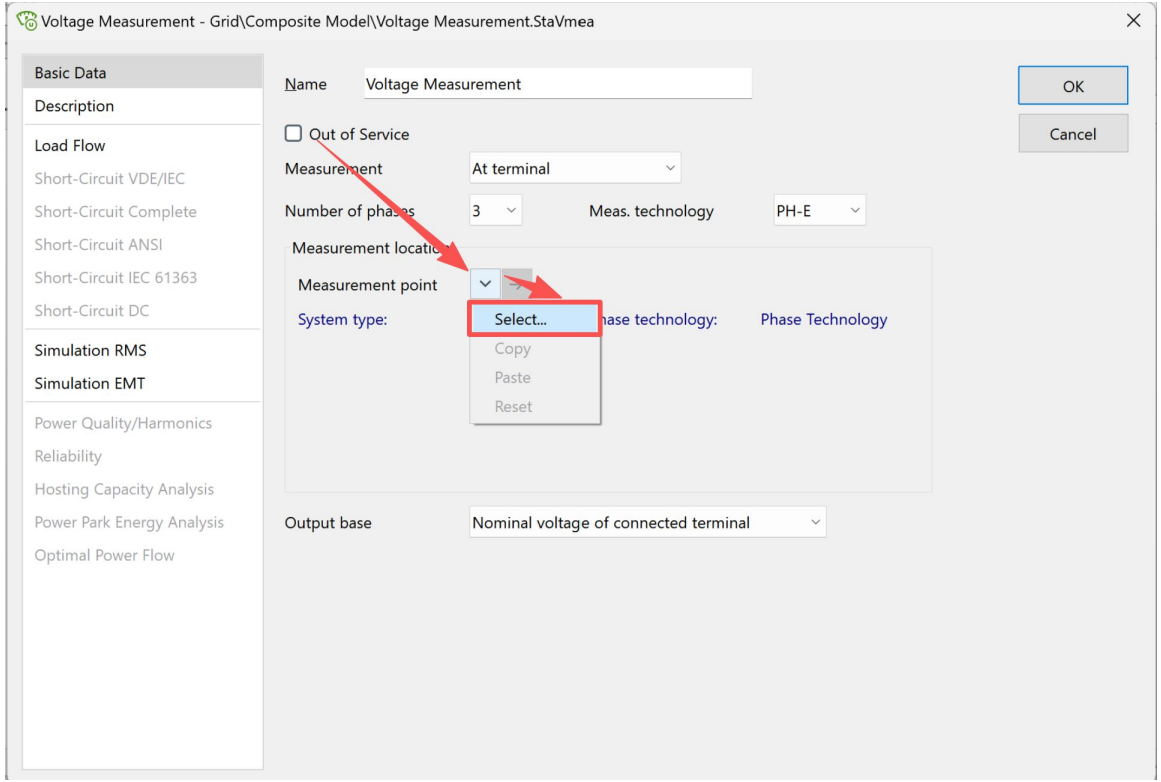
1. Enter a descriptive **Name** (e.g., *SCRXfmi*).
2. Ensure the **Type** points to the FMU-backed Modelica Model Type (e.g., *Dynamic Models\scrxFMI*).
3. Verify that the FMU information is correctly displayed (e.g., *FMI 2.0 – Model Exchange*).
4. Confirm settings and click **OK**.

At this stage, the FMU is fully integrated into PowerFactory as a Modelica Model within the Composite Model. This completes the link from the FMU file → Model Type → Composite Frame → Network Model, enabling FMI-based co-simulation with the grid.



1. In the **Composite Model** under *Network Data → Grid*, click **New Object**.
2. From the dialog, search for and select **Voltage Measurement**.
3. Confirm with **OK** to insert it into the composite model.

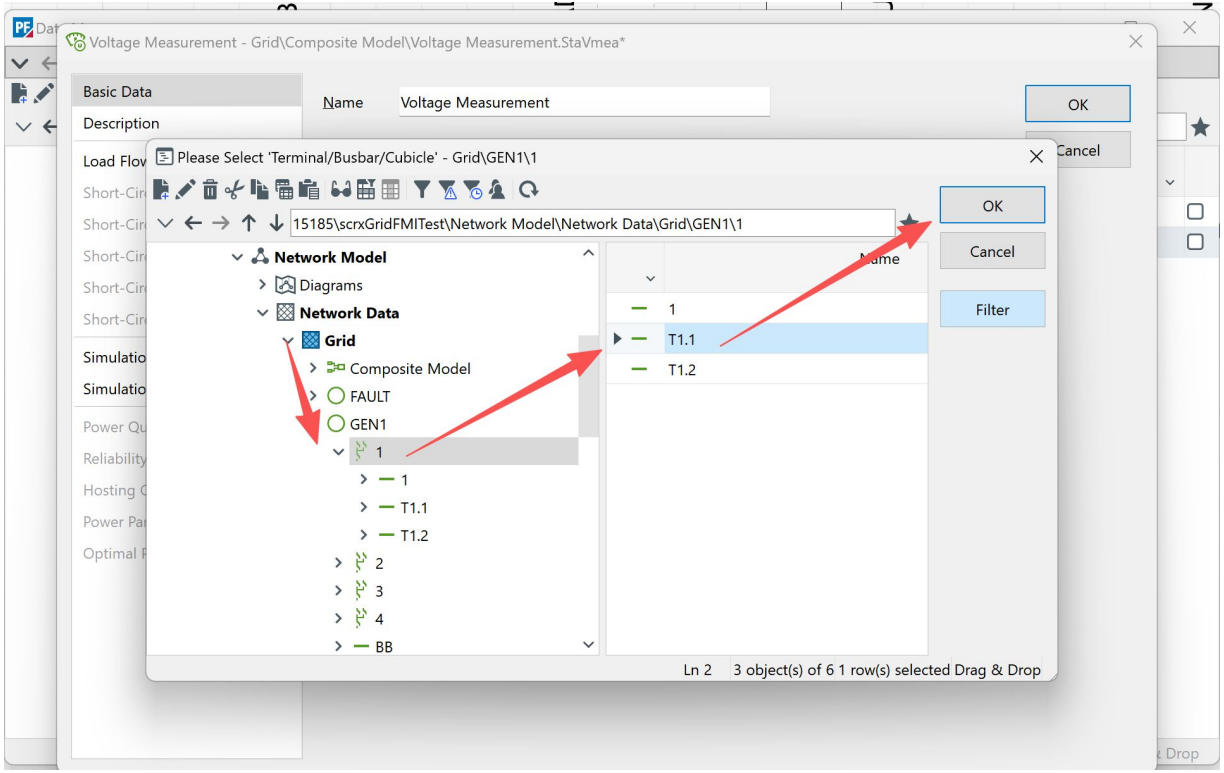
The Voltage Measurement element will allow simulation outputs (such as bus voltages) to be captured and analyzed, providing validation that the FMU and composite model are exchanging signals correctly.



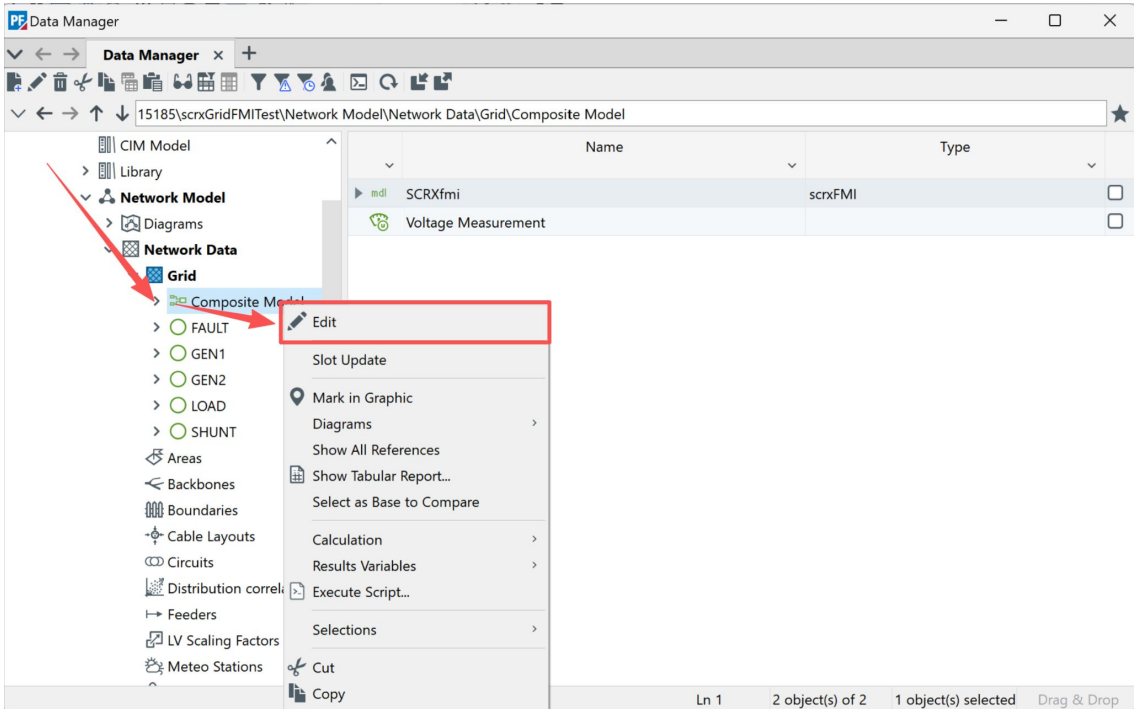
After creating the Voltage Measurement object, configure its parameters to monitor the correct location in the grid:

1. Set the **Measurement** to *At terminal* and define the **Number of phases** (e.g., *3*) and **Measurement technology** (e.g., *PH-E*).
2. Under **Measurement point**, click **Select…** to choose the terminal or bus where the measurement will be taken.
3. Leave the **Output base** as *Nominal voltage of connected terminal*, unless a different scaling is required.

This setup ensures that the measurement block records the terminal voltage during simulation. The results can then be used to validate the FMU's integration and assess its dynamic interaction with the network model.

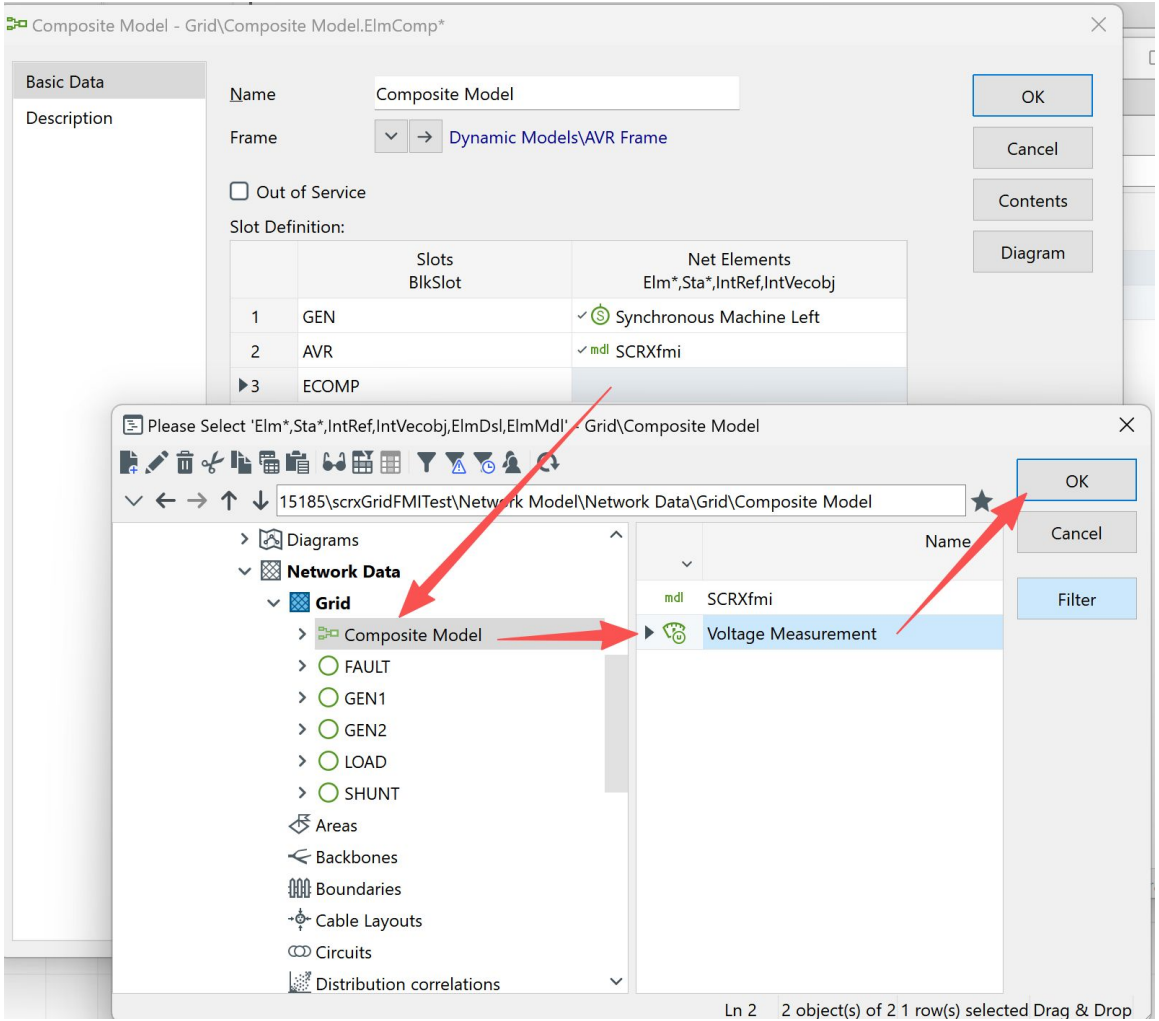


1. In the **Measurement point** selection dialog, expand the **Grid** and locate the desired generator (e.g., *GEN1*).
2. Choose the appropriate terminal (here: *T1.1*).
3. Confirm with **OK** to finalize the assignment.



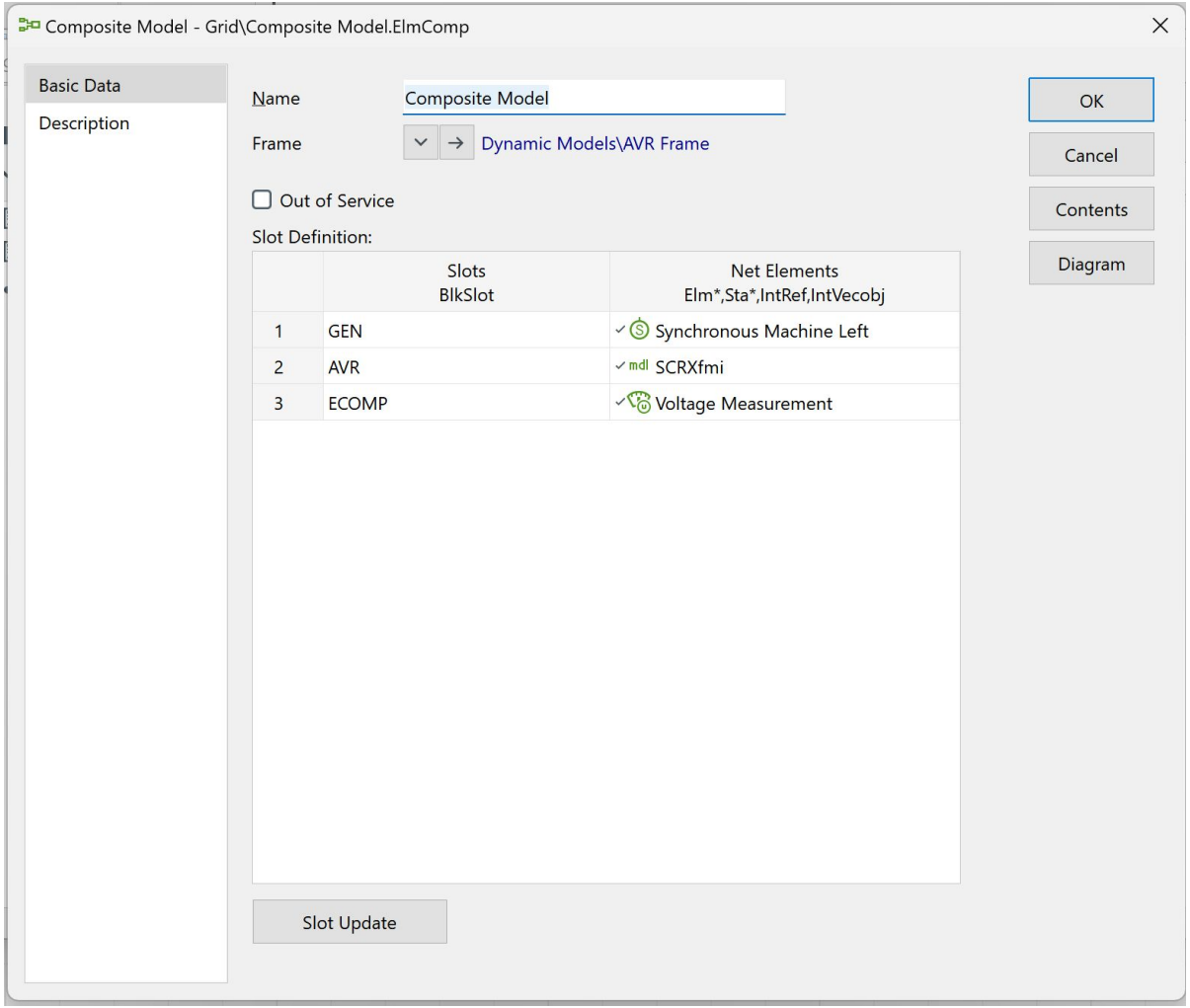
With the FMU and measurement objects added, the next step is to configure the Composite Model connections:

1. In the **Data Manager**, right-click on the **Composite Model**.
2. Select **Edit** from the context menu.



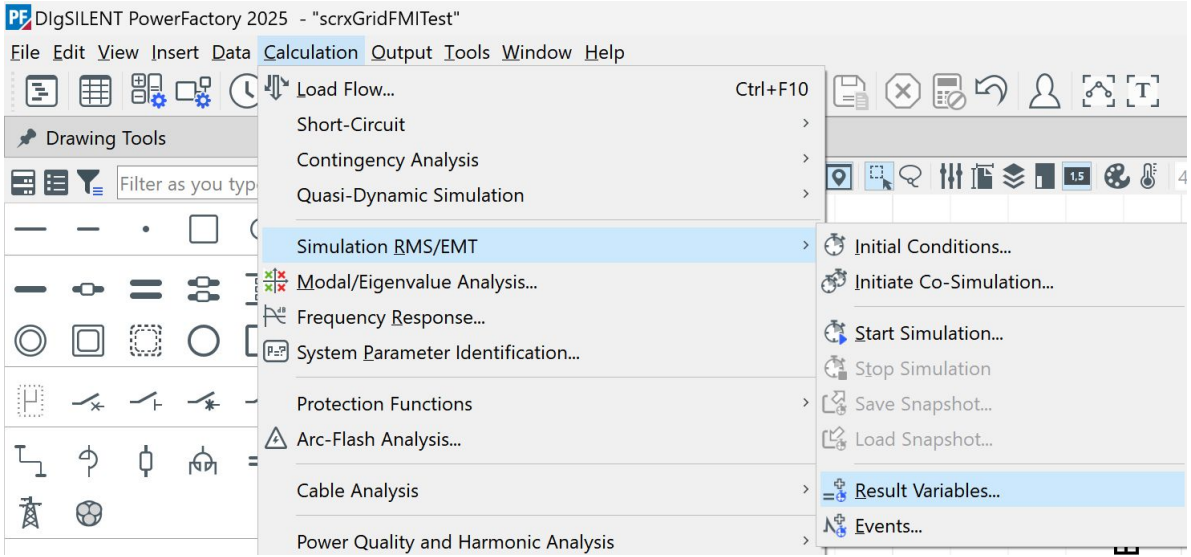
After entering the **Composite Model** editor, the final connections must be made:

1. In the **Slot Definition** panel, identify the slot to which the measurement is connected (e.g., AVR or ECOMP slot).
2. Under **Net Elements**, select the appropriate measurement device.
    - In this case, navigate to **Grid** → **Composite Model** → **Voltage Measurement** and confirm with **OK**.



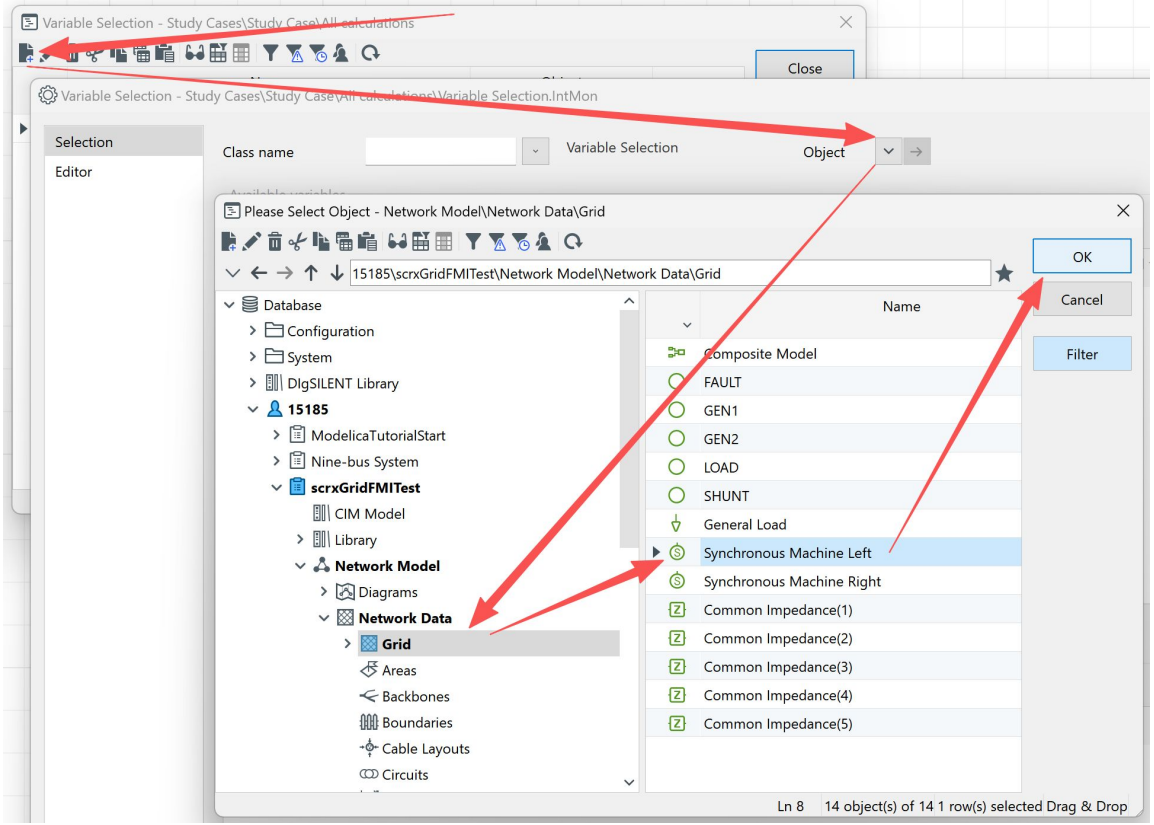
1. **GEN** → *Synchronous Machine Left*
2. **AVR** → *SCRXfmi* (imported FMU)
3. **ECOMP** → *Voltage Measurement*

At this point, the AVR frame is fully integrated into PowerFactory using the FMU exported from OpenModelica.



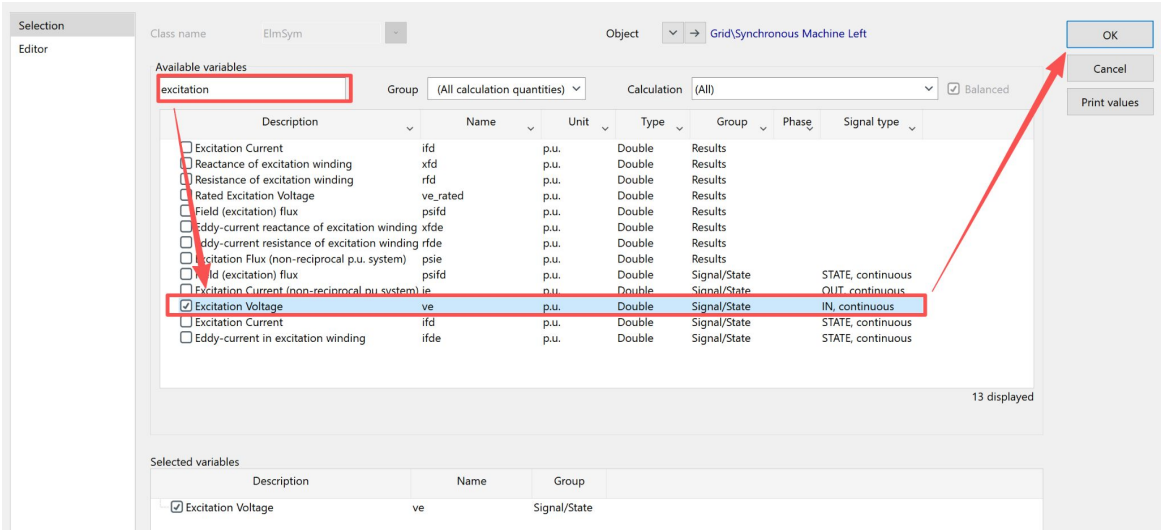
Once the composite model is configured, the final step is to perform a simulation and observe the AVR response.

1. Navigate to the **Calculation** menu.
2. Select **Simulation RMS/EMT** →
   - **Result Variables**: Choose which signals (e.g., terminal voltage, excitation voltage, AVR output) to record

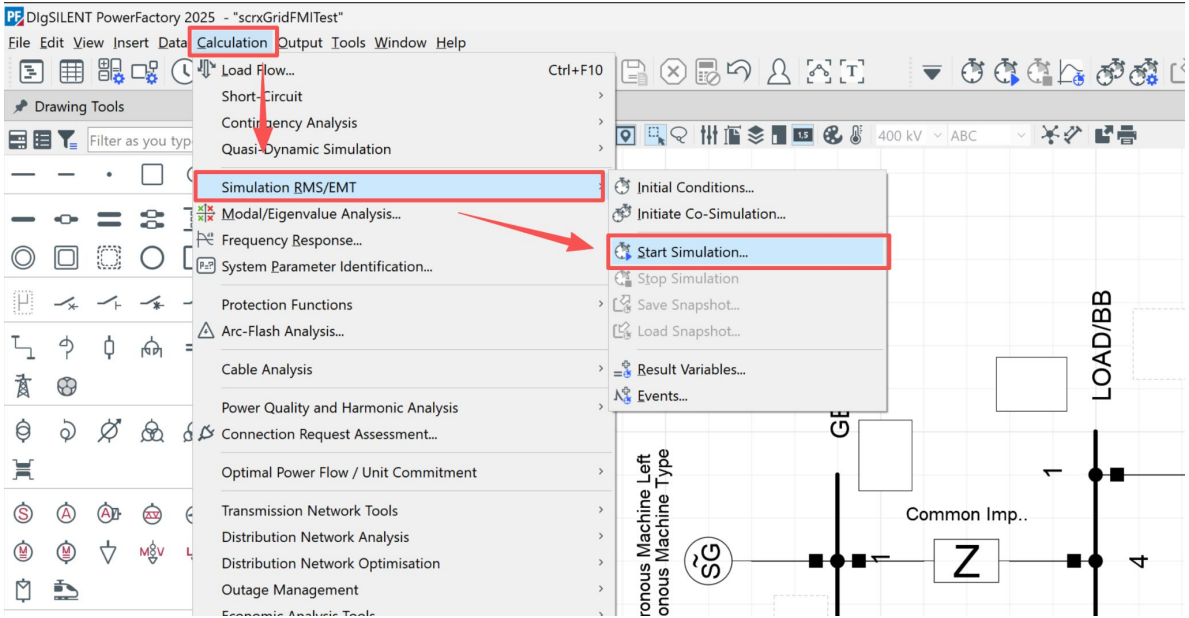


To observe the AVR and generator responses during the RMS/EMT simulation, you need to define which signals will be monitored.

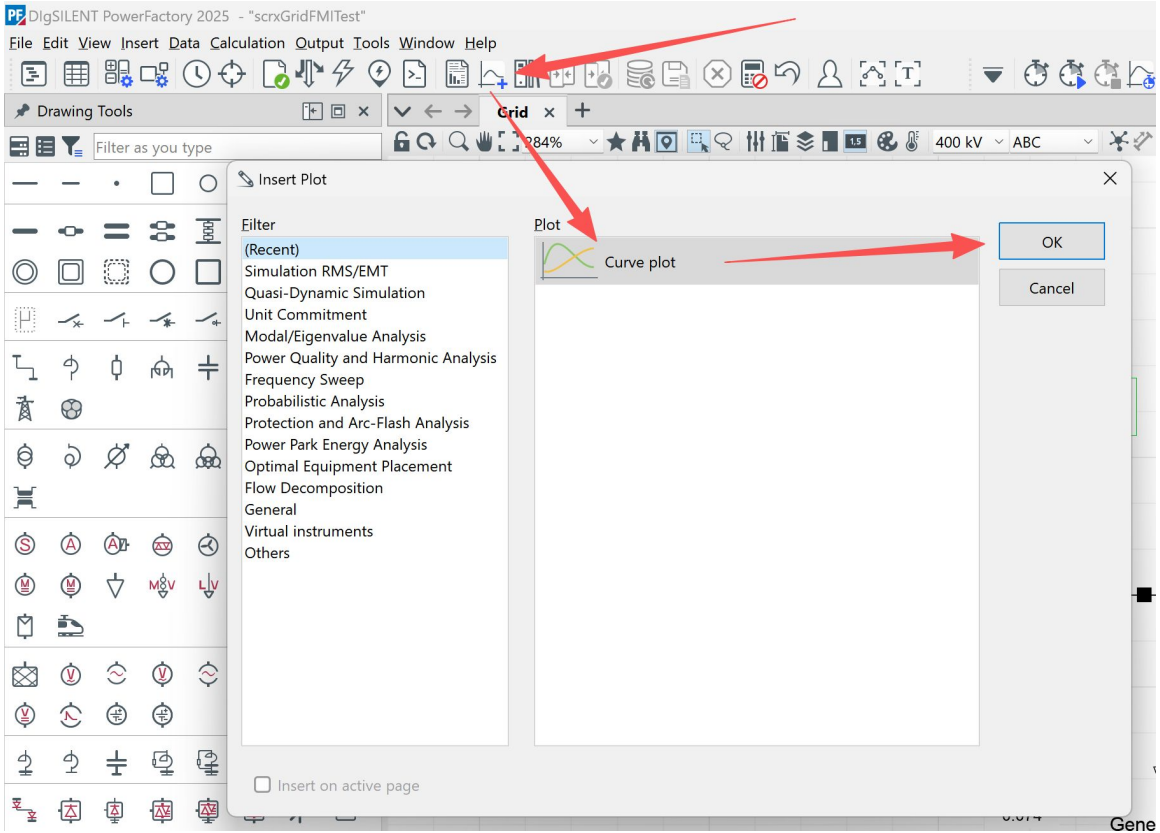
1. In the study case toolbar, open the **Variable Selection** menu.
2. Choose **Object** as the selection type.
3. From the network tree, navigate to your **Grid** → **Synchronous Machine Left** (or the machine connected to your AVR FMU).
4. Confirm with **OK**.



1. In the Variable Selection window, use the search bar to filter by typing “excitation”.
2. From the list of available variables, check Excitation Voltage (ve).
3. This variable represents the key AVR control signal.
4. Click OK to confirm the selection.

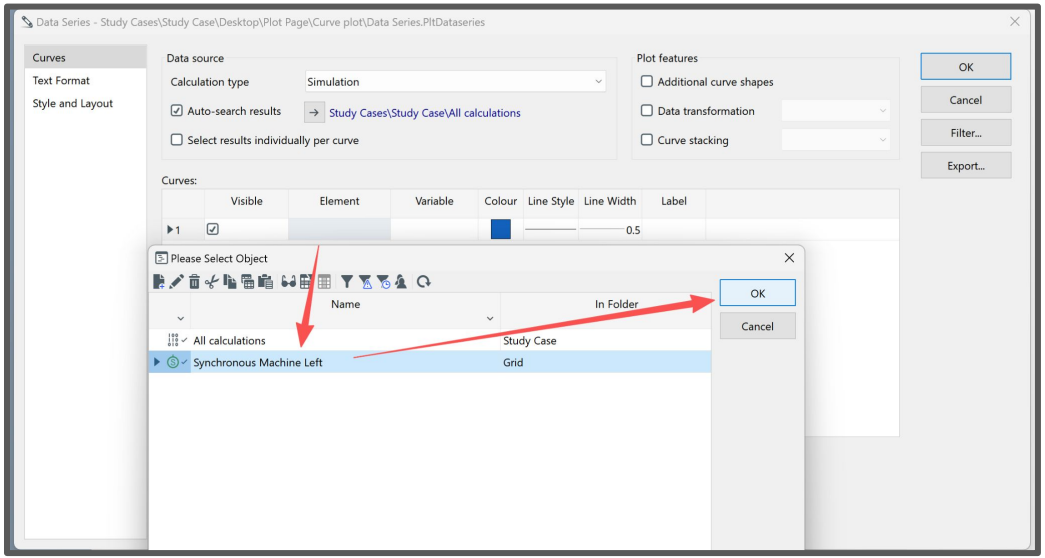


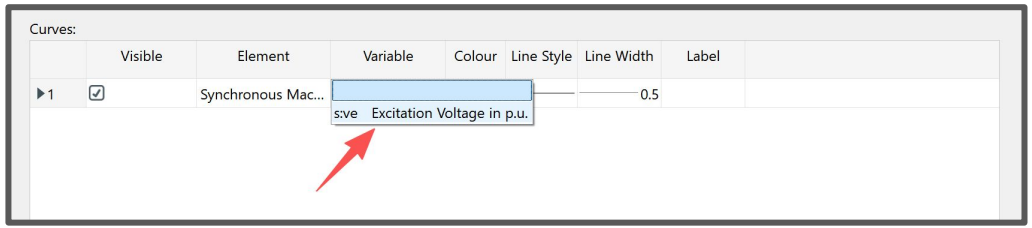
1. Go to the **Calculation** menu.
2. Select **Simulation RMS/EMT → Start Simulation**.
3. The simulation will run with the defined study case, recording the previously selected **excitation**.



Once the simulation has finished, you can visualize the results using PowerFactory's plotting tools.

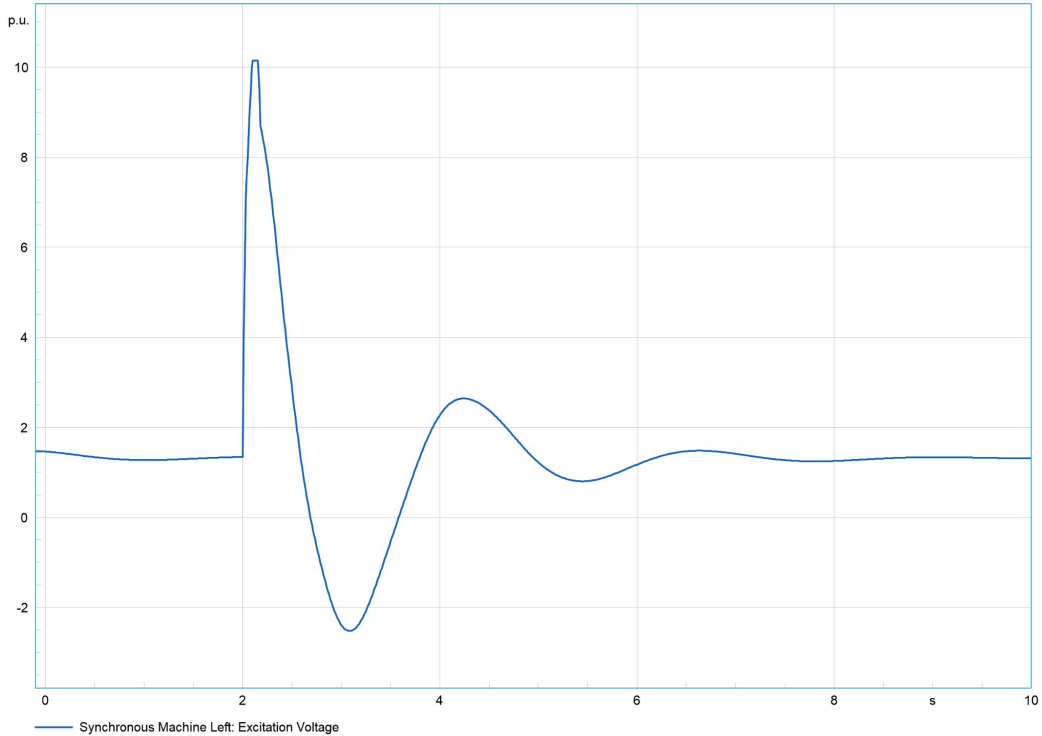
1. From the toolbar, click the **Insert Plot** button.
2. In the **Insert Plot** dialog, choose **Curve plot**.
3. Click **OK** to create a new plot window.





After inserting a curve plot, you must define which simulation results to display:

1. In the **Data Series** dialog, select the **calculation case** and the **network element** (e.g., *Synchronous Machine Left*).
   Under **Variable**, choose **Excitation Voltage in p.u.** (ve).
2. Confirm with **OK** to add the selected variable to the plot.



With this result, the tutorial is complete — you have successfully:

1. Exported an FMU from OpenModelica.
2. Imported and configured it in PowerFactory.
3. Built a composite AVR model.
4. Simulated and visualized the AVR’s response.